

claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf_56274ff0-603\agent-ac619656b95a85d7c.jsonl`
- SHA-256: `adc93f7d5c8c5239da19c6688c9cd077982fe9d3c2fba5e50276af02762db258`
- Source modified: `2026-06-21T02:36:38+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_56274ff0-603`
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`
```

Transcript

1. user (2026-06-21T02:34:25.860Z)

Map the TESTING surface so we can design a rigorous multi-profile strategy. Read `run_all_tests.py`, the `tests/` directory layout (`test_*`), `backend/eval/golden_fixtures.py` (the committed synthetic video-free regression fixtures = the \$0 CI gate / M0), the stub detector `seam` (`RALLY_DETECTOR_IMPL=stub / detector_impl=stub` for GPU-free e2e), any M3.5 parity tests, the storage backend fakes (in-memory `s3/gcs/gdrive` used by tests), and `.github/workflows/ci.yml` (the CI lanes: full suite, `ruff`, `mypy`, golden cross-OS). Identify the building blocks for a strategy that proves: the core is unaffected across profiles, the cloud path works (mocked + real-GPU), and nothing regresses. Tag `kind + reuse_status`.

2. assistant (2026-06-21T02:34:27.885Z)

I'll map the testing surface for the `badminton-highlight-indexer` project. Let me start by exploring the key files and directories you mentioned.

3. user (2026-06-21T02:34:28.341Z)

```
1      #!/usr/bin/env python
2      """Run the FULL test suite for the badminton-highlight-indexer.
3
4      A single, discoverable entry point so you (or a reviewer / CI) can verify the
5      whole suite before opening a PR:
6
7          python run_all_tests.py          # full suite, quiet
8          python run_all_tests.py -v      # verbose (passthrough to pytest)
9          python run_all_tests.py -k config # filter by keyword (passthrough)
10         python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
11
12     Anything after the script name is passed straight through to pytest.
13
14     Notes
15     -----
16     - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17     - The observability reporting tests skip cleanly if the optional obsreport
18       package isn't installed (they use pytest.importorskip). To exercise them too:
19         pip install -e ../sports-obsreport --no-build-isolation
20     - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21     """
22     from __future__ import annotations
23
24     import importlib.util
25     import subprocess
26     import sys
27
28
```

```

29     def _installed(module: str) -> bool:
30         try:
31             return importlib.util.find_spec(module) is not None
32         except (ImportError, ValueError):
33             return False
34
35
36     def main() -> int:
37         args = list(sys.argv[1:])
38         # Default to quiet unless the caller chose a verbosity.
39         if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
40             args = ["-q", *args]
41         cmd = [sys.executable, "-m", "pytest", *args]
42
43         obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests
will SKIP"
44         bar = "=" * 72
45         print(bar)
46         print(" Full test suite: " + " ".join(cmd))
47         print(f" obsreport (optional reporting dep): {obs}")
48         print(bar, flush=True)
49
50         rc = subprocess.run(cmd).returncode
51
52         print(bar)
53         print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}"))
54         print(bar)
55         return rc
56
57
58     if __name__ == "__main__":
59         raise SystemExit(main())
60

```

4. user (2026-06-21T02:34:28.838Z)

```

tests\__init__.py
tests\test_eval_batch.py
tests\test_eval_training.py
tests\test_annotation_providers.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\test_export_wasb_segments.py
tests\test_gemini_refine.py
tests\test_calibrate_local.py
tests\test_distill_local.py
tests\test_automated_provider.py
tests\test_eval_harness.py
tests\test_hit_detector.py
tests\test_hashing.py
tests\test_rally_slicer.py
tests\test_reporting_spec_faq.py
tests\test_eval_partial_labels.py
tests\test_detector_keying.py
tests\test_eval_metrics.py
tests\test_eval_annotations.py
tests\test_gt_loader.py
tests\test_import_smoke.py
tests\test_reporting_harvest.py
tests\test_run_signals.py
tests\test_eval_classifier.py
tests\test_eval_regression.py
tests\test_geometry.py
tests\test_path_safety.py

```

tests\test_providers.py
tests\test_rally_seq_proto.py
tests\test_reliability_hardening.py
tests\test_reporting.py
tests\test_sport_profiles.py
tests\test_video.py
tests\test_watermark.py
tests\test_cleanup_caches.py
tests\test_yolo_hybrid.py
tests\test_fusion_features.py
tests\test_rally_gate.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_logging_setup.py
tests\test_rally_summary.py
tests\test_eval_stats.py
tests\test_person_detector.py
tests\test_score_calibration.py
tests\test_execution_policy.py
tests\test_experiment_honest.py
tests\test_job_self_scheduling.py
tests\test_run_telemetry.py
tests\test_variant_tuning.py
tests\test_fusion_golden_telemetry.py
tests\test_per_user_gate.py
tests\test_per_user_eval.py
tests\test_per_user_regression.py
.claude\worktrees\bhi-execpolicy\tests__init__.py
.claude\worktrees\bhi-execpolicy\tests\test_ablation.py
.claude\worktrees\bhi-execpolicy\tests\test_annotation_providers.py
.claude\worktrees\bhi-execpolicy\tests\test_api_endpoints.py
.claude\worktrees\bhi-execpolicy\tests\test_async_jobs.py
.claude\worktrees\bhi-execpolicy\tests\test_audio_features.py
.claude\worktrees\bhi-execpolicy\tests\test_automated_provider.py
.claude\worktrees\bhi-execpolicy\tests\test_base.py
.claude\worktrees\bhi-execpolicy\tests\test_calibrate_local.py
.claude\worktrees\bhi-execpolicy\tests\test_calibrate_wasb.py
.claude\worktrees\bhi-execpolicy\tests\test_calibration.py
.claude\worktrees\bhi-execpolicy\tests\test_cleanup_caches.py
.claude\worktrees\bhi-execpolicy\tests\test_compiler.py
.claude\worktrees\bhi-execpolicy\tests\test_config.py
.claude\worktrees\bhi-execpolicy\tests\test_config_wasb.py
.claude\worktrees\bhi-execpolicy\tests\test_database.py
.claude\worktrees\bhi-execpolicy\tests\test_detector_keying.py
.claude\worktrees\bhi-execpolicy\tests\test_distill_local.py
.claude\worktrees\bhi-execpolicy\tests\test_eval_annotations.py
.claude\worktrees\bhi-execpolicy\tests\test_eval_batch.py
.claude\worktrees\bhi-execpolicy\tests\test_eval_classifier.py
.claude\worktrees\bhi-execpolicy\tests\test_eval_harness.py
.claude\worktrees\bhi-execpolicy\tests\test_eval_metrics.py
.claude\worktrees\bhi-execpolicy\tests\test_eval_partial_labels.py
.claude\worktrees\bhi-execpolicy\tests\test_eval_regression.py
.claude\worktrees\bhi-execpolicy\tests\test_eval_stats.py
.claude\worktrees\bhi-execpolicy\tests\test_eval_training.py
.claude\worktrees\bhi-execpolicy\tests\test_execution_policy.py
.claude\worktrees\bhi-execpolicy\tests\test_experiment.py
.claude\worktrees\bhi-execpolicy\tests\test_experiment_honest.py
.claude\worktrees\bhi-execpolicy\tests\test_export_wasb_segments.py
.claude\worktrees\bhi-execpolicy\tests\test_fusion_compare.py
.claude\worktrees\bhi-execpolicy\tests\test_fusion_features.py
.claude\worktrees\bhi-execpolicy\tests\test_fusion_golden_telemetry.py
.claude\worktrees\bhi-execpolicy\tests\test_fusion_hybrid.py
.claude\worktrees\bhi-execpolicy\tests\test_gemini.py
.claude\worktrees\bhi-execpolicy\tests\test_gemini_refine.py
.claude\worktrees\bhi-execpolicy\tests\test_geometry.py
.claude\worktrees\bhi-execpolicy\tests\test_gt_loader.py

```
.claude\worktrees\bhi-execpolicy\tests\test_hashing.py
.claude\worktrees\bhi-execpolicy\tests\test_hit_detector.py
.claude\worktrees\bhi-execpolicy\tests\test_import_smoke.py
.claude\worktrees\bhi-execpolicy\tests\test_job_self_scheduling.py
.claude\worktrees\bhi-execpolicy\tests\test_logging_setup.py
.claude\worktrees\bhi-execpolicy\tests\test_path_safety.py
.claude\worktrees\bhi-execpolicy\tests\test_per_user_eval.py
(Results are truncated. Consider using a more specific path or pattern.)
```

5. user (2026-06-21T02:34:28.930Z)

```
.claude\worktrees\bhi-execpolicy\.github\workflows\ci.yml
.claude\worktrees\agent-a771ff1d85e18f2ed\.github\workflows\ci.yml
.claude\worktrees\tier1-windowing\.github\workflows\ci.yml
.claude\worktrees\w2-safeguard\.github\workflows\ci.yml
.claude\worktrees\hardchop\.github\workflows\ci.yml
.claude\worktrees\boxhill17\.github\workflows\ci.yml
.github\workflows\ci.yml
.claude\worktrees\pr-stitch\.github\workflows\ci.yml
.claude\worktrees\gap-doc\.github\workflows\ci.yml
.claude\worktrees\resweep\.github\workflows\ci.yml
.claude\worktrees\recdoc\.github\workflows\ci.yml
```

6. user (2026-06-21T02:34:28.970Z)

```
backend\eval\__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\eval\gemini_refine.py
backend\eval\audio\__init__.py
backend\eval\audio\e1_probe.py
backend\eval\export_wasb_segments.py
backend\eval\metrics.py
backend\eval\regression.py
backend\eval\annotations.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\eval\eval_stats.py
backend\eval\score_calibration.py
backend\eval\batch.py
backend\eval\fusion_golden.py
backend\eval\per_user_gate.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py
.claude\worktrees\bhi-execpolicy\backend\eval\__init__.py
.claude\worktrees\bhi-execpolicy\backend\eval\ablation.py
.claude\worktrees\bhi-execpolicy\backend\eval\annotations.py
.claude\worktrees\bhi-execpolicy\backend\eval\audio\e1_probe.py
.claude\worktrees\bhi-execpolicy\backend\eval\audio\__init__.py
.claude\worktrees\bhi-execpolicy\backend\eval\batch.py
.claude\worktrees\bhi-execpolicy\backend\eval\calibrate_local.py
.claude\worktrees\bhi-execpolicy\backend\eval\calibrate_wasb.py
.claude\worktrees\bhi-execpolicy\backend\eval\calibration.py
.claude\worktrees\bhi-execpolicy\backend\eval\classifier.py
.claude\worktrees\bhi-execpolicy\backend\eval\distill_local.py
.claude\worktrees\bhi-execpolicy\backend\eval\eval_partial_labels.py
.claude\worktrees\bhi-execpolicy\backend\eval\eval_stats.py
.claude\worktrees\bhi-execpolicy\backend\eval\experiment.py
.claude\worktrees\bhi-execpolicy\backend\eval\export_wasb_segments.py
.claude\worktrees\bhi-execpolicy\backend\eval\fusion_audio.py
```

```

.claude\worktrees\bhi-execpolicy\backend\eval\fusion_compare.py
.claude\worktrees\bhi-execpolicy\backend\eval\fusion_golden.py
.claude\worktrees\bhi-execpolicy\backend\eval\gemini_refine.py
.claude\worktrees\bhi-execpolicy\backend\eval\gt_loader.py
.claude\worktrees\bhi-execpolicy\backend\eval\harness.py
.claude\worktrees\bhi-execpolicy\backend\eval\metrics.py
.claude\worktrees\bhi-execpolicy\backend\eval\per_user_eval.py
.claude\worktrees\bhi-execpolicy\backend\eval\per_user_gate.py
.claude\worktrees\bhi-execpolicy\backend\eval\per_user_regression.py
.claude\worktrees\bhi-execpolicy\backend\eval\rally_seq_proto.py
.claude\worktrees\bhi-execpolicy\backend\eval\regression.py
.claude\worktrees\bhi-execpolicy\backend\eval\score_calibration.py
.claude\worktrees\bhi-execpolicy\backend\eval\segmentation_metrics.py
.claude\worktrees\bhi-execpolicy\backend\eval\training.py
backend\eval\served_gate_a.py
backend\eval\distill_local.py
backend\eval\promotion.py
backend\eval\serve_contrast.py
backend\eval\splits.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\__init__.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\ablation.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\annotations.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\audio\__init__.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\audio\e1_probe.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\batch.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\calibrate_local.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\calibrate_wasb.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\calibration.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\classifier.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\distill_local.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\eval_partial_labels.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\eval_stats.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\experiment.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\export_wasb_segments.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\fusion_audio.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\fusion_compare.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\fusion_golden.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\gemini_refine.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\gt_loader.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\harness.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\metrics.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\per_user_eval.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\per_user_gate.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\per_user_regression.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\promotion.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\rally_seq_proto.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\regression.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\score_calibration.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\segmentation_metrics.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\serve_contrast.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\served_gate_a.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\splits.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\training.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval>windowing.py
backend\eval\segmentation_metrics.py
.claude\worktrees\tier1-windowing\backend\eval\ablation.py
.claude\worktrees\tier1-windowing\backend\eval\annotations.py
.claude\worktrees\tier1-windowing\backend\eval\audio\e1_probe.py
.claude\worktrees\tier1-windowing\backend\eval\audio\__init__.py
.claude\worktrees\tier1-windowing\backend\eval\batch.py
(Results are truncated. Consider using a more specific path or pattern.)

```

7. assistant (2026-06-21T02:34:31.754Z)

Now let me read the golden fixtures file and the CI workflow:

8. user (2026-06-21T02:34:32.562Z)

find: 'C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval': No such file or directory

9. user (2026-06-21T02:34:32.571Z)

```
1     name: CI
2
3     on:
4       push:
5         branches: [master]
6       pull_request:
7
8     permissions:
9       contents: read
10
11    concurrency:
12      group: ci-${{ github.ref }}
13      cancel-in-progress: true
14
15    jobs:
16      # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17      # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18      # tracked .py, including scripts/ and training/, which the backend-only
19      # py_compile smoke sweep does not reach.
20      lint:
21        name: Ruff lint
22        runs-on: ubuntu-latest
23        steps:
24          - uses: actions/checkout@v6
25          - uses: actions/setup-python@v6
26            with:
27              python-version: "3.13"
28          - name: Install ruff
29            run: python -m pip install ruff
30          - name: Ruff check
31            run: ruff check . --output-format github
32
33      # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34      # per-module quarantine list ? ratchet by removing entries as modules clean
35      # up. Typed core deps are installed so mypy sees their real signatures
36      # (without them ignore_missing_imports would Any-stub half the checks).
37      typecheck:
38        name: Mypy (lenient baseline)
39        runs-on: ubuntu-latest
40        steps:
41          - uses: actions/checkout@v6
42          - uses: actions/setup-python@v6
43            with:
44              python-version: "3.13"
45          - name: Install mypy + typed core deps
46            run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
47genai numpy
48          - name: Mypy
49            run: mypy backend training
50
51      # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
52      # module-level import error shipping while the suite stays green.
53      # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
54      # on machines without the GPU stack (test_import_smoke stubs what's absent).
55      import-smoke:
56        name: Syntax sweep + import smoke (no GPU stack)
57        runs-on: ubuntu-latest
58        steps:
59          - uses: actions/checkout@v6
```

```

59         - uses: actions/setup-python@v6
60         with:
61             python-version: "3.13"
62         - name: Install minimal deps
63           run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64         - name: Run import smoke tests
65           run: python -m pytest tests/test_import_smoke.py -v
66
67     full-suite:
68       name: Full test suite (offline, mocked)
69       runs-on: ubuntu-latest
70       steps:
71         - uses: actions/checkout@v6
72         - uses: actions/setup-python@v6
73         with:
74             python-version: "3.13"
75             cache: pip
76         - name: System libs for opencv-python
77           run: |
78             sudo apt-get update
79             sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
80         - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81           # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82           # need an out-of-band index that PEP 621 can't express. Keep installing
83           # them separately here, before the editable install pulls the rest.
84           run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85         - name: Install package (editable) + dev tooling
86           # Editable PEP 621 install: runtime deps + the `dev` group
87           # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88           run: python -m pip install -e .[dev]
89           # obsreport (private, soft dep) is absent here: reporting tests skip via
90           # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91           # is therefore calibrated against THIS lane's number (local runs with
92           # obsreport installed measure a little higher).
93           # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94           # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95           # without an owner decision.
96         - name: Run full suite (with coverage floor)
97           run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
99     # Cross-OS determinism guard for the committed golden regression fixtures
100    # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
101    # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
102    # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
103    # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
104    # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
105    golden-crossos:
106      name: Golden fixtures cross-OS (${ matrix.os })
107      strategy:
108        fail-fast: false
109        matrix:
110          os: [ubuntu-latest, windows-latest, macos-latest]
111      runs-on: ${ matrix.os }
112      steps:
113        - uses: actions/checkout@v6
114        - uses: actions/setup-python@v6
115        with:
116            python-version: "3.13"
117        - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
118          run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy

```

```

119         - name: Golden-fixtures characterization (cross-OS determinism guard)
120         # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
(synthetic /
121         # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
the
122         # de-attribution paths get cross-OS coverage and surface any win/mac float
drift.
123         run: |
124             python -m backend.eval.golden_fixtures --check
125             python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
126

```

10. assistant (2026-06-21T02:34:34.838Z)

Let me find the golden_fixtures.py file using PowerShell since the bash find didn't work on Windows:

11. user (2026-06-21T02:34:36.996Z)

Directory: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\boxhill17\backend\eval

Mode	LastWriteTime	Length
Name	-----	-----

-a---	6/17/2026 4:55 PM	57601 golden_fixtures.py

Directory: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gap-doc\backend\eval

Mode	LastWriteTime	Length
Name	-----	-----

-a---	6/18/2026 7:09 AM	57601 golden_fixtures.py

Directory: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\hardchop\backend\eval

Mode	LastWriteTime	Length
Name	-----	-----

-a---	6/16/2026 6:08 PM	57601 golden_fixtures.py

Directory: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\pr-stitch\backend\eval

Mode	LastWriteTime	Length
Name	-----	-----

-a---	6/17/2026 7:49 PM	57601 golden_fixtures.py

Directory: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\recdoc\backend\eval

Mode	LastWriteTime	Length
Name	-----	-----

-a---	6/18/2026 12:36 PM	57601 golden_fixtures.py

Directory: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\resweep\backend\eval

Mode Name	LastWriteTime	Length
----	-----	-----

-a---	6/18/2026 10:39 AM	57601 golden_fixtures.py

Directory: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\w2-safeguard\backend\eval

Mode Name	LastWriteTime	Length
----	-----	-----

-a---	6/16/2026 11:06 AM	31957 golden_fixtures.py

Directory: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval

Mode Name	LastWriteTime	Length
----	-----	-----

-a---	6/19/2026 1:03 PM	42641 golden_fixtures.py

12. user (2026-06-21T02:34:37.458Z)

```
1      """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3      `docs/GOLDEN_REGRESSION_FIXTURES.md` ($4, $6, $7). This module is the **single shared
4      replay code path** used by both the freeze tool and the characterization gate, and it
5      deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6      variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7      across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8      only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9      merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10     LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11     `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12     path simply never calls into it.)
13
14     The replay reuses the already-shipped eval stack VERBATIM::
15
16         parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17         ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18         ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20     Tier-0 / privacy invariants (D3, $4c, $6 of the doc), enforced here by construction:
21
22     * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
23     provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24     * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"``.
25     * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26     person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27     * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
```

```

28     passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29     named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30     silently churn fixtures.
31     * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32     the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33     takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34     has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35     ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36     tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38     $6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39     suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40     frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41     that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42     field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44     Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45     parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
46     NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47     the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48     never calls into it.
49     """
50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import re
56     import sys
57     from dataclasses import dataclass
58     from pathlib import Path
59     from typing import Any, Dict, List, Optional, Sequence, Tuple
60
61     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-----
62     from backend.eval import distill_local
63     from backend.eval.experiment import CONTROL, VideoCandidates, predict
64     from backend.eval.gt_loader import load_gt_intervals
65     from backend.eval.metrics import score
66     from backend.eval.regression import gt_hash
67
68     Interval = Tuple[float, float]
69
70     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
71     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
72     CURRENT_SCHEMA = 1
73
74     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
75     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
76     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"

```

```

77
78     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
79     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
80     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
81     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
82     #: re-states these in its provenance; the replay reads them from there.
83     PINNED_VT = 0.02
84     PINNED_MERGE_GAP = 2.0
85     PINNED_MIN_WINDOW = 2.0
86     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
87     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``_pinned_proposal``
88     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
89     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
90     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
91     PINNED_MAX_WINDOW = 0.0
92
93     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
94     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
95     SHUTTLE_FEATURES: Tuple[str, ...] = (
96         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
97     )
98     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
99         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
100     )
101
102     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
103     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
104     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
105
106     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
107     _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
108
109     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
110     #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
111     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
112
113     #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
114     #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
115     #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
116     #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
117     _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_-]*$")
118
119
120     # =====
121     # Synthetic trajectory generation (deterministic ? NO random)
122     # =====
123

```

```

124
125     @dataclass(frozen=True)
126     class RallySpec:
127         """One in-flight rally segment: the shuttle oscillates across the net midline.
128
129         ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
130         period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
131         net midline along the play axis. Deterministic sine motion ? fixed
132         velocity/crossings.
133         """
134         start_frame: int
135         n_frames: int
136         period: float = 30.0
137         amplitude: float = 400.0
138
139
140     @dataclass(frozen=True)
141     class GapSpec:
142         """Dead-time between rallies ? what separates one window from the next.
143
144         ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
145         ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
146         end an active run so the next rally forms a distinct window.
147         """
148         start_frame: int
149         n_frames: int
150         kind: str = "still" # "still" | "occluded"
151
152
153
154     @dataclass(frozen=True)
155     class FixtureSpec:
156         """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
157
158         ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
159         "labels.csv". For a synthetic fixture they line up with the rallies by construction,
160         which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
161         field-quality one (§6).
162         """
163         slug: str
164         description: str
165         fps: float
166         frame_width: float
167         frame_height: float
168         segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
169         gts: Tuple[Interval, ...]
170         net_midline: float = 640.0
171
172
173
174     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
175                      ) -> List[Tuple[int, int, float, float]]:
176         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
177         axis = x)."""
178         rows: List[Tuple[int, int, float, float]] = []
179         y_mid = frame_height / 2.0
180         for i in range(spec.n_frames):
181             x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
182             # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
183             unambiguously x.
184             y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
185             rows.append((spec.start_frame + i, 1, x, y))
186         return rows

```

```

186
187 def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
188                 ) -> List[Tuple[int, int, float, float]]:
189     """Dead-time rows: resting-visible (still) or absent (occluded)."""
190     rows: List[Tuple[int, int, float, float]] = []
191     y_mid = frame_height / 2.0
192     for i in range(spec.n_frames):
193         if spec.kind == "occluded":
194             rows.append((spec.start_frame + i, 0, 0.0, 0.0))
195         elif spec.kind == "still":
196             rows.append((spec.start_frame + i, 1, net_midline, y_mid))
197         else: # pragma: no cover - guarded by builder
198             raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'/'occluded')")
199     return rows
200
201
202 def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
203     """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
204     scenario."""
205     rows: List[Tuple[int, int, float, float]] = []
206     for seg in spec.segments:
207         if isinstance(seg, RallySpec):
208             rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
209         elif isinstance(seg, GapSpec):
210             rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
211         else: # pragma: no cover - guarded by dataclass shape
212             raise TypeError(f"unknown segment type {type(seg).__name__}")
213     rows.sort(key=lambda r: r[0])
214     return rows
215
216 def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
217 None:
218     """Write trajectory rows as the TrackNet CSV ``(Frame,Visibility,X,Y)``, LF + 6dp
219     floats."""
220     lines = ["Frame,Visibility,X,Y"]
221     for frame, vis, x, y in rows:
222         lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
223     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
224
225 def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
226     """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
227     LF."""
228     lines = ["start,end"]
229     for s, e in gts:
230         lines.append(f"{_fmt(s)},{_fmt(e)}")
231     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
232
233 # =====
234 # Built-in synthetic scenarios (the 3 committed slugs)
235 # =====
236
237 def _fr(seconds: float, fps: float) -> int:
238     return int(round(seconds * fps))
239
240
241 def builtin_specs() -> List[FixtureSpec]:
242     """The committed synthetic scenarios. Deterministic; chosen so each exercises a
243     distinct
244     windowing behaviour (one window / dead-time split / occlusion split)."""
245     fps = 30.0
246     fw, fh = 1280.0, 720.0

```

```

246         net = 640.0
247
248         # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
249         single = FixtureSpec(
250             slug="clean_single_rally",
251             description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
252             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
253             segments=(RallySpec(start_frame=0, n_frames=120),),
254             # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
255             gts=((0.0, 4.0),),
256         )
257
258         # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
259         #     low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
260         r0 = RallySpec(start_frame=0, n_frames=120)
261         gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
262         r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
263         multi = FixtureSpec(
264             slug="multi_rally_with_deadtime",
265             description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
266             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
267             segments=(r0, gap_still, r1),
268             gts=((0.0, 4.0), (7.0, 11.0)),
269         )
270
271         # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
272         #     visibility break ? proves occlusion (not just low velocity) separates windows.
273         o0 = RallySpec(start_frame=0, n_frames=120)
274         gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
275         o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
276         occ = FixtureSpec(
277             slug="occlusion_break",
278             description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
279             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
280             segments=(o0, gap_occ, o1),
281             gts=((0.0, 4.0), (7.0, 11.0)),
282         )
283
284         return [single, multi, occ]
285
286
287     def _spec_by_slug() -> Dict[str, FixtureSpec]:
288         return {s.slug: s for s in builtin_specs()}
289
290
291     # =====
292     # The shared replay (CONTROL / pure-stdlib path)
293     # =====
294
295
296     def _fmt(x: float) -> str:
297         """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
298         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
299         if r == 0.0:
300             r = 0.0
301         return f"{r:.6f}"
302
303

```

```

304     def _round(x: float) -> float:
305         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
306         return 0.0 if r == 0.0 else r
307
308
309     def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
310         """The windowing proposal tuple `(vt, mg, mwd)`, read from the fixture provenance
when present
311         (the pinned params travel with the fixture), else the module-level pinned defaults.
NEVER
312         `windowing.SERVED`. NOTE: the provenance `max_win` (W1) is intentionally NOT
read here ?
313         `build_candidates` takes only this 3-tuple, so the replay characterizes the W1-OFF
path; the
314         pinned `max_win` is a recorded annotation, not a knob the gate applies (see
PINNED_MAX_WINDOW)."""
315         if prov and "windowing" in prov:
316             w = prov["windowing"]
317             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
318         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
319
320
321     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
322         """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.
323
324         Resolves `slug_or_dir` (a slug string or a fixture directory `Path`) to its
committed
325         `trajectory.csv` + `labels.csv` + `provenance.json`, then:
326
327         parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
328         cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
329
330         Returns `{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
331         control_segments:[[s,e]], score:{...}, gt_hash}`. CONTROL keeps every candidate
then
332         merges overlaps; NO numpy is touched.
333         """
334         fdir = _resolve_dir(slug_or_dir)
335         slug = fdir.name
336         traj_csv = fdir / "trajectory.csv"
337         labels_csv = fdir / "labels.csv"
338         prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
339
340         fps, fw = _fps_fw(prov, slug)
341         iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
342         proposal = _pinned_proposal(prov)
343
344         intervals, feature_rows = distill_local.build_candidates(
345             str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
346
347         # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
348         # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
349         # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
350         features: Dict[str, List[float]] = {
351             name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
352         }
353         gts = load_gt_intervals(str(labels_csv), strict=True)
354         vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
355

```

```

356         preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
357         sc = score(preds, gts, iou)
358
359         candidates: List[Dict[str, Any]] = []
360         for j, (s, e) in enumerate(intervals):
361             shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
362             # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
363             shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
364             candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
365
366         return {
367             "slug": slug,
368             "schema_version": CURRENT_SCHEMA,
369             "iou": _round(iou),
370             "candidates": candidates,
371             "control_segments": [[_round(s), _round(e)] for s, e in preds],
372             "score": {
373                 "iou_threshold": _round(sc.iou_threshold),
374                 "num_pred": int(sc.num_pred),
375                 "num_gt": int(sc.num_gt),
376                 "true_positives": int(sc.true_positives),
377                 "false_positives": int(sc.false_positives),
378                 "false_negatives": int(sc.false_negatives),
379                 "precision": _round(sc.precision),
380                 "recall": _round(sc.recall),
381                 "f1": _round(sc.f1),
382                 "mean_iou": _round(sc.mean_iou),
383                 "mean_start_error": _round(sc.mean_start_error),
384                 "mean_end_error": _round(sc.mean_end_error),
385             },
386             "gt_hash": gt_hash(gts),
387         }
388
389
390         # =====
391         # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
392         # =====
393
394
395     def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
396                     ) -> Dict[str, Any]:
397         """The committed provenance block ? pins the windowing + privacy/licensing tags
($4c, $5)."""
398         desc = ""
399         if isinstance(spec_or_dir, FixtureSpec):
400             desc = spec_or_dir.description
401         return {
402             "slug": slug,
403             "schema_version": CURRENT_SCHEMA,
404             "kind": "synthetic",
405             "license": "synthetic",
406             "minors_free": True,
407             "description": desc,
408             "fps": _round(fps),
409             "frame_width": _round(fw),
410             "iou": _round(iou),
411             "windowing": {
412                 "vt": PINNED_VT,
413                 "mg": PINNED_MERGE_GAP,
414                 "mwd": PINNED_MIN_WINDOW,
415                 "max_win": PINNED_MAX_WINDOW,

```



```

416         },
417         "paths": {
418             "trajectory": "trajectory.csv",
419             "labels": "labels.csv",
420             "expected": "expected.json",
421         },
422         "note": (
423             "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
424             "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
($6)."
425         ),
426     }
427
428
429     def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
430                       iou: float = 0.5) -> Dict[str, Any]:
431         """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
COMMITTED
432         ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).
433
434         Idempotent: rewrites the same bytes given the same committed inputs. Requires the
trajectory
435         + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
436         """
437         fdir = fixtures_dir / slug
438         traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
439         if not traj_csv.is_file() or not labels_csv.is_file():
440             raise FileNotFoundError(
441                 f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
442
443         # Provenance first (it pins the windowing the replay reads), then replay ? expected.
444         spec = _spec_by_slug().get(slug)
445         fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
446         prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
447         _write_json(fdir / "provenance.json", prov)
448
449         expected = replay_fixture(fdir)
450         _write_json(fdir / "expected.json", expected)
451         return expected
452
453
454     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
455         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
456         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
457         specs = builtin_specs()
458         fixtures_dir.mkdir(parents=True, exist_ok=True)
459         slugs: List[str] = []
460         for spec in specs:
461             fdir = fixtures_dir / spec.slug
462             fdir.mkdir(parents=True, exist_ok=True)
463             rows = make_synthetic_trajectory(spec)
464             write_trajectory_csv(rows, fdir / "trajectory.csv")
465             write_labels_csv(spec.gts, fdir / "labels.csv")
466             freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
467             slugs.append(spec.slug)
468         _write_manifest(specs, fixtures_dir)
469         return slugs
470
471
472     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
473                       fixtures_dir: Path = FIXTURES_DIR) -> List[str]:

```

```

474         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
475         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
regression; the
476         mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
477         if all_slugs:
478             targets = [d.name for d in sorted(fixture_dir.iterdir())
479                        if d.is_dir() and (d / "trajectory.csv").is_file()]
480         elif slug:
481             targets = [slug]
482         else:
483             raise ValueError("refresh_snapshot needs --slug S or --all")
484         for s in targets:
485             freeze_fixture(s, fixture_dir=fixture_dir)
486         return targets
487
488
489     # =====
490     # Manifest entry derivation (shared ? used by both the synthetic and real-fixture
writers)
491     # =====
492
493
494     def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
495         """A manifest row for a single fixture, derived from its provenance block."""
496         slug = prov["slug"]
497         return {
498             "slug": slug,
499             "fps": _round(float(prov["fps"])),
500             "frame_width": _round(float(prov["frame_width"])),
501             "windowing": dict(prov.get("windowing", {})),
502             "kind": prov.get("kind", "synthetic"),
503             "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
504             "paths": {
505                 "dir": slug,
506                 "trajectory": f"{slug}/trajectory.csv",
507                 "labels": f"{slug}/labels.csv",
508                 "expected": f"{slug}/expected.json",
509                 "provenance": f"{slug}/provenance.json",
510             },
511         }
512
513
514     def _append_manifest_entry(prov: Dict[str, Any], fixture_dir: Path) -> None:
515         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
516         entries = load_manifest(fixture_dir)
517         entries = [e for e in entries if e.get("slug") != prov["slug"]]
518         entries.append(_manifest_entry_for(prov))
519         entries.sort(key=lambda e: str(e.get("slug")))
520         _write_json(fixture_dir / "manifest.json", entries)
521
522
523     # =====
524     # Manifest + loaders
525     # =====
526
527
528     def _write_manifest(specs: Sequence[FixtureSpec], fixture_dir: Path) -> None:
529         entries = [{
530             "slug": spec.slug,
531             "fps": _round(spec.fps),
532             "frame_width": _round(spec.frame_width),
533             "windowing": {

```

```

534         "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
535         "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
536     },
537     "kind": "synthetic",
538     "schema_version": CURRENT_SCHEMA,
539     "paths": {
540         "dir": spec.slug,
541         "trajectory": f"{spec.slug}/trajectory.csv",
542         "labels": f"{spec.slug}/labels.csv",
543         "expected": f"{spec.slug}/expected.json",
544         "provenance": f"{spec.slug}/provenance.json",
545     },
546 } for spec in specs]
547 _write_json(fixtures_dir / "manifest.json", entries)
548
549
550 def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
551     """Load the fixtures manifest (returns [] if absent)."""
552     path = fixtures_dir / "manifest.json"
553     if not path.is_file():
554         return []
555     data = _read_json(path)
556     if not isinstance(data, list): # pragma: no cover - corrupt manifest
557         raise ValueError(f"{path}: manifest must be a JSON array")
558     return data
559
560
561 def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
562     """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
563     whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
the
564     ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
forward-version
565     bump in one is never masked by an in-range value in the other (red-team #7)."""
566     fdir = fixtures_dir / slug
567     prov = _read_json(fdir / "provenance.json")
568     expected = _read_json(fdir / "expected.json")
569     for which, blob in (("expected", expected), ("provenance", prov)):
570         sv = int(blob.get("schema_version", 0))
571         if sv > CURRENT_SCHEMA:
572             raise ValueError(
573                 f"{slug}: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "
574                 f"upgrade the harness")
575     return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
576
577
578 # =====
579 # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
580 # =====
581
582
583 def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
584     """Return a list of human-readable mismatch messages (empty == match).
585
586     Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
587     TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
588     equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
589     safe (the repo runs with core.autocrlf=true).
590

```

```

591     Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
592     per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
593     a fixed-field loop alone would ignore an added field (red-team #5)."""
594     out: List[str] = []
595
596     def exact(path: str, a: Any, b: Any) -> None:
597         if a != b:
598             out.append(f"{path}: {a!r} != {b!r} (exact)")
599
600     def approx(path: str, a: Any, b: Any) -> None:
601         try:
602             if abs(float(a) - float(b)) > _FLOAT_TOL:
603                 out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
604         except (TypeError, ValueError):
605             out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
606
607     # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
608     if set(recomputed) != set(committed):
609         out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")
610
611     exact("slug", recomputed.get("slug"), committed.get("slug"))
612     exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
613     exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
614     approx("iou", recomputed.get("iou"), committed.get("iou"))
615
616     rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
617     if len(rc) != len(cc):
618         out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
619     else:
620         for i, (r, c) in enumerate(zip(rc, cc)):
621             approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
622             approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
623             rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
624             if set(rs) != set(cs):
625                 out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
626         else:
627             for k in rs:
628                 if k == "net_crossings":
629                     exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
630                 else:
631                     approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
632
633     rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
634     if len(rseg) != len(cseg):
635         out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
636     else:
637         for i, (r, c) in enumerate(zip(rseg, cseg)):
638             approx(f"control_segments[{i}][0]", r[0], c[0])
639             approx(f"control_segments[{i}][1]", r[1], c[1])
640
641     rsc, csc = recomputed.get("score", {}), committed.get("score", {})
642     # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
643     if set(rsc) != set(csc):
644         out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
645     int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
646     float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",

```

```

647         "mean_start_error", "mean_end_error")
648     for k in int_fields:
649         exact(f"score.{k}", rsc.get(k), csc.get(k))
650     for k in float_fields:
651         approx(f"score.{k}", rsc.get(k), csc.get(k))
652
653     return out
654
655
656 # =====
657 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
658 # =====
659
660
661 def _write_json(path: Path, obj: Any) -> None:
662     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
663     path.write_text(text + "\n", encoding="utf-8", newline="\n")
664
665
666 def _read_json(path: Path) -> Any:
667     with open(path, encoding="utf-8") as f:
668         return json.load(f)
669
670
671 def _resolve_dir(slug_or_dir: Any) -> Path:
672     if isinstance(slug_or_dir, Path):
673         return slug_or_dir
674     p = Path(str(slug_or_dir))
675     if p.is_dir():
676         return p
677     return FIXTURES_DIR / str(slug_or_dir)
678
679
680 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
681     if prov and "fps" in prov and "frame_width" in prov:
682         return float(prov["fps"]), float(prov["frame_width"])
683     spec = _spec_by_slug().get(slug)
684     if spec is not None:
685         return spec.fps, spec.frame_width
686     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
687
688
689 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
690     prov_path = fdir / "provenance.json"
691     prov = _read_json(prov_path) if prov_path.is_file() else None
692     return _fps_fw(prov, slug)
693
694
695 # =====
696 # CLI
697 # =====
698
699
700 def _cmd_check(fixtures_dir: Path) -> int:
701     manifest = load_manifest(fixtures_dir)
702     if not manifest:
703         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
704         return 1
705     failures = 0
706     for entry in manifest:
707         slug = entry["slug"]
708         committed = _read_json(fixtures_dir / slug / "expected.json")
709         recomputed = replay_fixture(fixtures_dir / slug)

```

```

710         mismatches = compare_expected(recomputed, committed)
711         if mismatches:
712             failures += 1
713             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
714                   f"(characterization = behaviour-locking, NOT correctness ? $6):")
715             for m in mismatches:
716                 print(f"    - {m}")
717         else:
718             sc = recomputed["score"]
719             print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
720                   f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
721         print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
722               f\"(synthetic-tier = plumbing correctness, not field quality ? $6)\")
723         return 0 if failures == 0 else 1
724
725
726     def main(argv: Optional[Sequence[str]] = None) -> int:
727         ap = argparse.ArgumentParser(
728             description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
729 harness.")
730         g = ap.add_mutually_exclusive_group(required=True)
731         g.add_argument("--freeze-synthetic", action="store_true",
732                        help="generate synthetic trajectory+labels for all built-in
733 scenarios, then "
734                            "freeze expected.json+provenance.json + write manifest.json")
735         g.add_argument("--refresh-snapshot", action="store_true",
736                        help="re-freeze expected.json from COMMITTED trajectory+labels (no
737 regeneration)")
738         g.add_argument("--check", action="store_true",
739                        help="replay all fixtures + parse-compare vs committed expected.json
740 (exit 0/1)")
741         g.add_argument("--freeze-real", action="store_true",
742                        help="(P1) de-attribute an OWNED real trajectory+labels into a
743 committable Tier-0 "
744                            "fixture; REFUSED without --owned --minors-free and a non-blank
745 --license")
746         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
747 slug for "
748                            "--freeze-real (default: a random fx_<hex> id)")
749         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
750 snapshot")
751         ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
752 (review the diff!)")
753         # --freeze-real (P1 / M2 refusal gate) options:
754         ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
755 (Frame,Visibility,X,Y)")
756         ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
757 (start,end)")
758         ap.add_argument("--license", dest="license_", default=None,
759                        help="--freeze-real) the source license tag (required, non-blank)")
760         ap.add_argument("--owned", action="store_true",
761                        help="--freeze-real) assert owner-recorded Fork-A source
762 (required)")
763         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
764                        help="--freeze-real) assert the source contains NO minors
765 (required)")
766         ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
767         ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
768 real) source frame width (px)")
769         ap.add_argument("--label", default="cleared_real",
770                        help="--freeze-real) non-identifying scenario label ? STRICT slug
771 only "
772                            "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
773         ap.add_argument("--source-note", dest="source_note", default="",

```

```

759             help="--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
760             "attribution back-channel, so it never reaches the committed
provenance "
761             "(scanned defensively, then dropped). Retained for CLI
compatibility.")
762         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
763             help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
764         ap.set_defaults(time_origin_reset=True)
765         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
766             help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
767         args = ap.parse_args(list(argv) if argv is not None else None)
768
769         fixtures_dir: Path = args.fixtures_dir
770
771         if args.freeze_synthetic:
772             slugs = freeze_synthetic(fixtures_dir)
773             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
'.join(slugs)}")
774             print(f"  -> {fixtures_dir}")
775             return 0
776
777         if args.refresh_snapshot:
778             if not args.slug and not args.all:
779                 ap.error("--refresh-snapshot requires --slug S or --all")
780             # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
real
781             # regression, so the deliberate, recorded reason is a required arg, not a soft
warning.
782             if not (args.reason and args.reason.strip()):
783                 ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
stamp a real "
784                     "regression; record the deliberate reason and review the visible
diff ($6).")
785             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
786             print(f"[golden-fixtures] refreshed snapshot for: {'', '.join(targets)}")
787             f" (reason: {args.reason})")
788             return 0
789
790         if args.freeze_real:
791             # The real-fixture subsystem (P1 + M2) lives in golden_real_fixtures (split
out); import it
792             # here so the CLI reaches the SAME freeze_real_fixture / RefusalError it always
has.
793             from backend.eval import golden_real_fixtures
794             missing = [n for n, v in (("--trajectory", args.trajectory),
795                                     ("--labels", args.labels),
796                                     ("--fps", args.fps),
797                                     ("--frame-width", args.frame_width)) if v is None]
798             if missing:
799                 ap.error("--freeze-real requires " + ", ".join(missing))
800             try:
801                 expected = golden_real_fixtures.freeze_real_fixture(
802                     args.label, args.trajectory, args.labels,
803                     license=args.license_ if args.license_ is not None else "",
804                     minors_free=args.minors_free, owned=args.owned,
805                     fps=args.fps, frame_width=args.frame_width,
806                     fixtures_dir=fixtures_dir, slug=args.slug,
807                     time_origin_reset=args.time_origin_reset, source_note=args.source_note)
808             except golden_real_fixtures.RefusalError as e:
809                 # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-

```

```

zero.
810         print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
811         return 2
812         print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
"
813             f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
814         print(" S6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
815             "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
816         return 0
817
818     # --check
819     return _cmd_check(fixtures_dir)
820
821
822     # =====
823     # Re-exports ? the real-fixture subsystem (P1 + M2) lives in ``golden_real_fixtures``
but
824     # is exposed here so every historical ``golden_fixtures.<name>`` import path stays
valid.
825     # =====
826
827     #: The names the real-fixture module owns; re-exported lazily (PEP 562 ``__getattr__``)
so the
828     #: ``golden_fixtures`` ? ``golden_real_fixtures`` cycle resolves regardless of import
order ?
829     #: ``golden_real_fixtures`` imports the shared primitives FROM this module, so an EAGER
bottom-of-
830     #: file re-export would fail when ``golden_real_fixtures`` is imported FIRST (its own
names would
831     #: not yet be defined when this module's import of them runs). The lazy hook defers the
import to
832     #: first attribute access, by which point BOTH modules are fully initialized.
833     _REAL_FIXTURE_EXPORTS: Tuple[str, ...] = (
834         "RefusalError", "freeze_real_fixture", "_scan_forbidden", "_new_surrogate_slug",
835         "_reset_trajectory_rows", "_reset_labels", "_SCHEMA_FILENAME_LITERALS",
836     )
837
838
839     def __getattr__(name: str) -> Any:
840         """Lazily resolve the re-exported real-fixture names from ``golden_real_fixtures``
(PEP 562)."""
841         if name in _REAL_FIXTURE_EXPORTS:
842             from backend.eval import golden_real_fixtures
843             return getattr(golden_real_fixtures, name)
844             raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
845
846
847     if __name__ == "__main__":
848         raise SystemExit(main())
849

```

13. assistant (2026-06-21T02:34:43.355Z)

Now let me search for test patterns related to detector stubs and storage backends:

14. user (2026-06-21T02:34:44.682Z)

Found 5 files

tests\test_storage.py

tests\test_wasb_cache.py

backend\pipeline\detectors\wasb_infer.py

backend\eval\experiment.py

tests\test_personalization_serving.py

15. user (2026-06-21T02:34:44.683Z)

Found 21 files

backend\config\models.py
tests\test_native_wasb_runner.py
backend\pipeline\detectors\native_wasb_runner.py
tests\test_worker.py
artifacts\so.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\worker__main__.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
tests\test_stub_runner.py
backend\pipeline\detectors\stub_runner.py
tests\test_motion_segmenter.py
tests\test_job_worker_module_seam.py
tests\test_fusion_compare.py
tests\test_tracknet_hybrid.py
tests\test_job_worker_loop.py
tests\test_fusion_golden_telemetry.py
backend\pipeline\detectors\person_detector.py
tests\test_import_smoke.py
tests\test_automated_provider.py
backend\annotations\automated_provider.py

16. user (2026-06-21T02:34:44.919Z)

Found 17 files

backend\main.py
tests\test_api_endpoints.py
tests\test_workspace.py
backend\config\models.py
tests\test_native_wasb_runner.py
backend\pipeline\detectors\native_wasb_runner.py
artifacts\so.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\worker__main__.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\stub_runner.py
tests\test_experiment.py
backend\pipeline\detectors\tracknet_runner.py
tests\test_async_jobs.py
tests\test_reingest_safety.py
backend\eval\fusion_golden.py

17. assistant (2026-06-21T02:34:46.677Z)

Let me read the key test files and check the test structure:

18. user (2026-06-21T02:34:47.125Z)

```
1      """CPU stub detector runner ? the "mock a GPU with a CPU" regression seam.
2
3      Proves the trajectory pipeline runs end-to-end with NO GPU/WSL: the stub emits a
4      deterministic Frame,Visibility,X,Y CSV that parses + windows through the real
5      (model-agnostic) helpers, and the `_resolve_runner` hook swaps it in via config/env
6      while leaving the default (real WSL runner) path untouched.
7      """
8      import os
9
10     import pytest
```

```

11
12 from backend.pipeline.detectors.base import DetectorRunner
13 from backend.pipeline.detectors.stub_runner import StubDetectorRunner, StubConfig
14 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
15
16
17 # --- the runner honours the DetectorRunner contract, on CPU, no GPU/WSL -----
18
19 def test_stub_is_a_detector_runner_and_healthcheck_ok():
20     runner = StubDetectorRunner()
21     assert isinstance(runner, DetectorRunner)
22     ok, msg = runner.healthcheck()
23     assert ok is True
24     assert "stub" in msg.lower()
25
26
27 def test_run_predict_writes_valid_trajectory_csv(tmp_path):
28     runner = StubDetectorRunner()
29     csv_path = runner.run_predict("C:/videos/match.mp4", str(tmp_path),
video_id="abc123def456ghi")
30     assert csv_path and os.path.exists(csv_path)
31     # MD5-namespaced filename (same contract as the real runners).
32     assert os.path.basename(csv_path) == "match__abc123def456_stub.csv"
33     with open(csv_path) as f:
34         header = f.readline().strip()
35         assert header == "Frame,Visibility,X,Y"
36
37
38 def test_synthetic_trajectory_yields_one_window_per_rally(tmp_path):
39     runner = StubDetectorRunner(StubConfig(num_rallies=2))
40     csv_path = runner.run_predict("clip.mp4", str(tmp_path), video_id="deadbeefcafe00")
41     points = runner.parse_trajectory_csv(csv_path)
42     assert any(p.visible for p in points) and any(not p.visible for p in points)
43     windows = runner.trajectory_to_action_windows(
44         points, fps=30.0, frame_width=1280.0,
45         velocity_thresh=0.02, merge_gap=2.0, min_window_duration=2.0,
46     )
47     # Two in-flight bursts separated by a >merge_gap dead gap => two candidate windows.
48     assert len(windows) == 2
49     assert all(isinstance(w, dict) for w in windows)
50
51
52 def test_run_predict_is_deterministic(tmp_path):
53     a = StubDetectorRunner().run_predict("x.mp4", str(tmp_path / "a"), video_id="v1")
54     b = StubDetectorRunner().run_predict("x.mp4", str(tmp_path / "b"), video_id="v1")
55     assert open(a, "rb").read() == open(b, "rb").read()
56
57
58 def test_replay_mode_emits_provided_csv_verbatim(tmp_path):
59     src = tmp_path / "golden.csv"
60     src.write_text("Frame,Visibility,X,Y\n0,1,100.0,200.0\n1,1,112.0,200.0\n")
61     runner = StubDetectorRunner(StubConfig(replay_csv=str(src)))
62     out = runner.run_predict("any.mp4", str(tmp_path / "out"), video_id="zz")
63     assert open(out).read() == src.read_text()
64
65
66 # --- the injection hook: stub swaps in via config/env, default path unchanged ---
67
68 _REAL_SENTINEL = object()
69
70
71 class _FakeTraj(TrajectoryHybridSegmenter):
72     family = "wasb"
73     display_label = "Fake"
74

```

```

75     @property
76     def name(self): # pragma: no cover - trivial
77         return "fake_hybrid"
78
79     @property
80     def description(self): # pragma: no cover - trivial
81         return "fake trajectory segmenter for the resolve-hook test"
82
83     def _build_runner(self, idx_cfg):
84         return _REAL_SENTINEL, {"k": "v"}
85
86
87     @pytest.fixture
88     def seg():
89         return _FakeTraj(None, {})
90
91
92     def test_resolve_runner_default_delegates_to_build_runner(seg, monkeypatch):
93         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
94         runner, params = seg._resolve_runner({})
95         assert runner is _REAL_SENTINEL and params == {"k": "v"}
96
97
98     def test_resolve_runner_config_selects_stub(seg, monkeypatch):
99         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
100         runner, params = seg._resolve_runner({"detector_impl": "stub"})
101         assert isinstance(runner, StubDetectorRunner)
102         assert params == {"detector_impl": "stub"}
103
104
105     def test_resolve_runner_env_overrides_config(seg, monkeypatch):
106         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
107         # Even with config asking for the real runner, the env var wins.
108         runner, _ = seg._resolve_runner({"detector_impl": "auto"})
109         assert isinstance(runner, StubDetectorRunner)
110
111
112     # --- the CPU "mock GPU" trajectory for training: label-consistent synthetic path ---
113
114     def test_synth_trajectory_from_labels(tmp_path):
115         from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
116         labels = tmp_path / "v.rallies.csv"
117
118         labels.write_text("rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
119                          "1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
120         out = tmp_path / "traj.csv"
121         synth_trajectory_from_labels(str(labels), str(out), fps=30.0, frame_width=1280.0)
122
123         pts = StubDetectorRunner().parse_trajectory_csv(str(out))
124         # every visible frame falls inside a labeled rally window; gaps are invisible
125         vis_times = [p.frame / 30.0 for p in pts if p.visible]
126         assert vis_times and all((2.0 <= t < 5.0) or (9.0 <= t < 12.0) for t in vis_times)
127         # windowing recovers the two rallies
128         wins = StubDetectorRunner().trajectory_to_action_windows(
129             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
130             min_window_duration=1.0)
131         assert len(wins) == 2

```

19. user (2026-06-21T02:34:47.595Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     The S3/GCS backends are exercised with injected FAKE clients (no boto3/google-cloud-
storage,

```

```

4      no network); the gdrive backend reads a locally MOUNTED Drive, exercised against a
tmp_path
5      that stands in for the mount ? all offline, matching the repo's ethos.
6      """
7      import os
8      import subprocess
9      import sys
10
11     import pytest
12
13     from backend import storage
14     from backend.storage import StorageBackend, fetch, get_storage, list_uris, parse_uri,
put
15     from backend.storage.local import LocalStorage
16     from backend.storage.s3 import S3Storage
17     from backend.storage.gcs import GCSStorage
18     from backend.storage.gdrive import GDriveStorage
19
20
21     # ----- URI parsing
22
23     @pytest.mark.parametrize("uri,backend,bucket,key", [
24         ("/data/x.mp4", "local", "", "/data/x.mp4"),
25         ("rel/x.mp4", "local", "", "rel/x.mp4"),
26         ("C:/Users/a/x.mp4", "local", "", "C:/Users/a/x.mp4"),      # Windows path: no
':://'
27         ("C:\\Users\\a\\x.mp4", "local", "", "C:\\Users\\a\\x.mp4"),
28         ("local:///abs/x.mp4", "local", "", "/abs/x.mp4"),
29         ("s3://rally/videos/m.mp4", "s3", "rally", "videos/m.mp4"),
30         ("gs://rally-eu/w/model.json", "gcs", "rally-eu", "w/model.json"), # gs:// -> gcs
31         ("gcs://rally/x", "gcs", "rally", "x"),
32         ("gdrive://Sharing/clip.mp4", "gdrive", "", "Sharing/clip.mp4"), # path under My
Drive (no bucket)
33     ])
34     def test_parse_uri(uri, backend, bucket, key):
35         ref = parse_uri(uri)
36         assert (ref.backend, ref.bucket, ref.key) == (backend, bucket, key)
37
38
39     def test_parse_uri_rejects_unknown_scheme():
40         with pytest.raises(ValueError):
41             parse_uri("ftp://host/x")
42
43
44     # ----- local backend
45
46     def test_local_fetch_is_identity_passthrough(tmp_path):
47         src = tmp_path / "v.mp4"
48         src.write_bytes(b"abc")
49         # No dst => returns the source path unchanged (no copy): preserves today's
behaviour.
50         assert fetch(str(src)) == str(src)
51
52
53     def test_local_put_and_roundtrip(tmp_path):
54         src = tmp_path / "in.csv"
55         src.write_text("Frame,Visibility,X,Y\n")
56         dst_uri = str(tmp_path / "out" / "copy.csv")
57         put(str(src), dst_uri)
58         assert (tmp_path / "out" / "copy.csv").read_text() == src.read_text()
59
60
61     def test_local_exists_stat_list_delete(tmp_path):
62         be = LocalStorage()
63         (tmp_path / "a.txt").write_text("hello")

```

```

64     (tmp_path / "sub").mkdir()
65     (tmp_path / "sub" / "b.txt").write_text("hi")
66     assert be.exists(parse_uri(str(tmp_path / "a.txt")))
67     assert not be.exists(parse_uri(str(tmp_path / "nope.txt")))
68     assert be.stat(parse_uri(str(tmp_path / "a.txt"))).size == 5
69     listed = sorted(r.key for r in be.list(parse_uri(str(tmp_path))))
70     assert listed == sorted([str(tmp_path / "a.txt"), str(tmp_path / "sub" / "b.txt")])
71     be.delete(parse_uri(str(tmp_path / "a.txt")))
72     assert not (tmp_path / "a.txt").exists()
73
74
75     def test_local_signed_url_raises():
76         with pytest.raises(NotImplementedError):
77             LocalStorage().signed_url(parse_uri("/x"))
78
79
80     def test_list_uris_helper(tmp_path):
81         (tmp_path / "x.txt").write_text("1")
82         (tmp_path / "y.txt").write_text("2")
83         uris = sorted(list_uris(str(tmp_path)))
84         assert all(u.startswith("local://") for u in uris)
85         assert len(uris) == 2
86
87
88     # ----- registry / get_storage
89
90     def test_get_storage_defaults_to_local():
91         assert isinstance(get_storage(), LocalStorage)
92         assert isinstance(get_storage(config={"backend": "local"}), LocalStorage)
93
94
95     def test_get_storage_unknown_backend_raises():
96         with pytest.raises(ValueError):
97             get_storage("azure")
98
99
100     def test_gdrive_not_mounted_raises_with_remedy(monkeypatch):

```

20. user (2026-06-21T02:34:47.627Z)

```

1     """Tests for the experiment harness (backend.eval.experiment).
2
3     Pure/offline ? synthetic candidate rows, no GPU/WSL. The load-bearing test is the
4     **no-regression invariant** (`test_disabled_cues_byte_identical_to_control`): a variant
5     that merely carries a disabled cue must predict exactly what control predicts. That is
the
6     guarantee that lets new cues be merged freely (`EXPERIMENT_HARNESS.md` §6).
7     """
8     import json
9     import os
10
11     import pytest
12
13     from backend.eval.classifier import LogisticRegression
14     from backend.eval.experiment import (
15         CONTROL,
16         ExperimentError,
17         Variant,
18         VideoCandidates,
19         _is_fusion_schema,
20         compare,
21         compare_unlabeled,
22         evaluate_variant,
23         load_golden_videos,
24         load_video_candidates,

```

```

25         predict,
26         record_experiment,
27     )
28     from backend.infrastructure.database import Database
29
30
31     # A video whose third candidate is a held-shuttle false fire (net_crossings=0): it does
32     # not overlap any golden rally, so it is a negative by construction.
33     def _held_shuttle_video(name="v", gts=None):
34         return VideoCandidates(
35             name=name,
36             intervals=[(0.0, 10.0), (20.0, 30.0), (12.0, 13.0)],
37             features={"net_crossings": [4.0, 3.0, 0.0]},
38             gts=gts if gts is not None else [(0.0, 10.0), (20.0, 30.0)],
39         )
40
41
42     # Three videos where feature "x" perfectly separates rallies (x=5) from junk (x=0),
43     # so an honest LOVO learned variant should recover F1?1.0 on each held-out video.
44     def _separable_videos():
45         vids = []
46         for k in range(3):
47             base = 100.0 * k
48             vids.append(VideoCandidates(
49                 name=f"sep{k}",
50                 intervals=[(base + 0, base + 10), (base + 20, base + 30),
51                     (base + 40, base + 41), (base + 50, base + 51)],
52                 features={"x": [5.0, 5.0, 0.0, 0.0]},
53                 gts=[(base + 0, base + 10), (base + 20, base + 30)],
54             ))
55         return vids
56
57
58     # ----- golden-features
59     schema
60     def test_is_fusion_schema_distinguishes_lean_and_full():
61         """The shared schema predicate keeps full-fusion rows (shuttle+person, audio
62         optional) and
63         rejects the leaner velocity-windowing rows (no cue dicts); an empty candidate list
64         is not a
65         fusion file. Single source of truth for the three golden-feature loaders
66         (ablation.load_videos / fusion_compare.load_videos /
67         fusion_audio.load_videos_with_audio)."""
68         full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
69         no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}] #
70         audio_optional
71         lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
72         assert _is_fusion_schema(full) is True
73         assert _is_fusion_schema(no_audio) is True
74         assert _is_fusion_schema(lean) is False
75         assert _is_fusion_schema([]) is False
76
77     def _write_no_audio_golden(d, name, shuttle_cols, person_cols):
78         """A FULL-FUSION-SCHEMA golden file (shuttle + person cue dicts) but NO audio block
79         ? the
80         schema that pre-dates the audio augment. The three loaders treat it differently by
81         policy."""
82         cand = {"start": 0.0, "end": 10.0,
83             "shuttle": {fn: 1.0 for fn in shuttle_cols},
84             "person": {fn: 0.0 for fn in person_cols},
85             "label": 1}
86         data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
87             "candidates": [cand], "gts": [[0.0, 10.0]]}

```

```

83         with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
84             json.dump(data, f)
85
86
87     def test_golden_loaders_audio_policy_differs(tmp_path):
88         """Pin the ONE behavioural difference between the three structurally-identical
golden-feature
89         loaders: their AUDIO POLICY, on the SAME audio-less full-fusion file.
90
91         - ``ablation.load_videos``          TOLERATES missing audio ? defaults the columns
to 0.0;
92         - ``fusion_audio.load_videos_with_audio``  REQUIRES audio ? raises ``SystemExit``;
93         - ``fusion_compare.load_videos``  OMITS audio entirely ? loads, no audio
columns.
94
95         Column source also differs (ablation's hardcoded ``SHUTTLE_COLUMNS`` vs the fusion
modules'
96         ``SHUTTLE_FEATURE_NAMES`` derived from ``distill_local.FEATURE_NAMES``) but the two
match
97         today ? this test writes both column sets so the audio-policy contrast is
isolated."""
98         from backend.eval import ablation, fusion_audio, fusion_compare
99
100        d = str(tmp_path)

```

21. assistant (2026-06-21T02:34:49.705Z)

Let me check the test directory more comprehensively and look for conf test patterns:

22. user (2026-06-21T02:34:50.064Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\tests'

23. user (2026-06-21T02:34:52.416Z)

ls: cannot access 'C:\Users\avidu\Projects\badminton-highlight-indexer\tests': No such file or directory

24. user (2026-06-21T02:34:52.563Z)

No files found

25. user (2026-06-21T02:34:53.055Z)

No files found

26. assistant (2026-06-21T02:34:54.555Z)

Let me try with correct PowerShell syntax:

27. user (2026-06-21T02:34:56.111Z)

```

__init__.py
test_ablation.py
test_annotation_providers.py
test_api_endpoints.py
test_async_jobs.py
test_audio_features.py
test_automated_provider.py
test_base.py
test_calibrate_local.py
test_calibrate_wasb.py

```

```
test_calibration.py
test_cleanup_caches.py
test_compiler.py
test_config_wasb.py
test_config.py
test_court_gate.py
test_database.py
test_detector_keying.py
test_distill_local.py
test_eval_annotations.py
```

28. user (2026-06-21T02:34:56.115Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

29. assistant (2026-06-21T02:34:57.729Z)

Let me check what conftest files exist:

30. user (2026-06-21T02:34:59.213Z)

(PowerShell completed with no output)

31. assistant (2026-06-21T02:35:02.212Z)

Let me check pytest configuration and look at several test files:

32. user (2026-06-21T02:35:02.672Z)

```
1      # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2      #
3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4      # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-__init__'d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
11     dependencies]
12     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
13     [build-system]
14     requires = ["hatchling"]
15     build-backend = "hatchling.build"
16
17     [project]
18     name = "badminton-highlight-indexer"
19     version = "0.1.0"
20     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
21     readme = "README.md"
22     requires-python = ">=3.10"
23     license = { text = "MIT" }
24     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
25     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
26
27     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
28     dependencies = [
29         "fastapi>=0.111.0",
30         "uvicorn>=0.30.1",
31         "opencv-python>=4.9.0.80",
32         "numpy>=1.26.4",
33         "pydantic>=2.7.1",
34         "jinja2>=3.1.4",
```



```

34     "python-dotenv>=1.0.0",
35     "google-genai>=2.4.0",
36     "supervision>=0.21.0,<0.30.0",
37 ]
38
39 [project.optional-dependencies]
40 # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41 dev = [
42     "pytest",
43     "pytest-cov",
44     "httpx",          # fastapi TestClient transport
45     "ruff",
46     "mypy",
47 ]
48
49 # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50 # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51 # fetched over SSH; if you lack access, simply omit this group ? reporting
52 # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53 #   pip install -e ../sports-obsreport --no-build-isolation
54 reporting = [
55     "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56 ]
57
58 # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59 # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60 # express, so they are NOT declarable here. Install them out-of-band instead:
61 #   # CUDA 12.4 box:
62 #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
63 #   https://download.pytorch.org/whl/cu124
64 #   # CPU-only box (also what CI uses):
65 #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
66 gpu = []
67
68 # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
69 # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
70 # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
71 # DO Spaces + MinIO via endpoint_url.) `gdrive` needs NO extra ? it reads a locally
72 # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
73 backend/storage/gdrive.py.
74 storage-s3 = ["boto3>=1.34"]
75 storage-gcs = ["google-cloud-storage>=2.16"]
76
77 [project.scripts]
78 # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
79 indexer = "backend.main:main"
80
81 [tool.hatch.metadata]
82 # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
83 # reference*); hatchling requires opting in to allow those in metadata.
84 allow-direct-references = true
85
86 [tool.hatch.build.targets.wheel]
87 # `backend` is a namespace package (no __init__.py); list both import roots
88 # explicitly so an editable/wheel install exposes `import backend` and
89 # `import training` regardless of cwd.
90 packages = ["backend", "training"]

```

33. user (2026-06-21T02:35:03.022Z)

```

pytest.ini
.claude\worktrees\bhi-execpolicy\pytest.ini
.claude\worktrees\agent-a771ff1d85e18f2ed\pytest.ini

```

```
.claude\worktrees\tier1-windowing\pytest.ini
.claude\worktrees\w2-safeguard\pytest.ini
.claude\worktrees\hardchop\pytest.ini
.claude\worktrees\boxhill17\pytest.ini
.claude\worktrees\pr-stitch\pytest.ini
.claude\worktrees\gap-doc\pytest.ini
.claude\worktrees\resweep\pytest.ini
.claude\worktrees\recdoc\pytest.ini
```

34. user (2026-06-21T02:35:03.046Z)

No files found

35. user (2026-06-21T02:35:05.405Z)

```
1      [pytest]
2      # When `pytest` is invoked without an explicit path, only collect from tests/.
3      # scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
4      # and should never run as part of the regular test suite.
5      testpaths = tests
6
```

36. user (2026-06-21T02:35:05.842Z)

```
1      """Tests for the regression runner (GS-2): baseline store, regress(), the
2      provider-driven end-to-end path, and the CLI exit-code contract."""
3      import pytest
4
5      from backend.infrastructure.database import Database
6      from backend.eval import regression
7      import run_regression
8
9
10     @pytest.fixture
11     def db(tmp_path):
12         d = Database(str(tmp_path / "t.db"))
13         d.add_video("vid1", "/x/m.mp4", "processed", [])
14         # Two confirmed segments matching the GT below ? perfect baseline.
15         d.add_play_segment("vid1", 0.0, 10.0)
16         d.add_play_segment("vid1", 20.0, 30.0)
17         return d
18
19
20     GT = [(0.0, 10.0), (20.0, 30.0)]
21
22
23     # ----- baseline store -----
24
25     def test_save_and_load_baseline_roundtrip(tmp_path):
26         p = str(tmp_path / "b.json")
27         regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
28         b = regression.load_baselines(p)
29         assert b["vid1::final"]["precision"] == 1.0
30
31
32     def test_load_baselines_absent_is_empty(tmp_path):
33         assert regression.load_baselines(str(tmp_path / "nope.json")) == {}
34
35
36     # ----- regress() -----
37
38     def test_no_baseline_is_not_a_regression(db, tmp_path):
39         r = regression.regress(db, "vid1", GT, baseline_path=str(tmp_path / "b.json"))
40         assert r.regressed is False
```

```

41     assert r.precision == 1.0 and r.recall == 1.0
42     assert any("no baseline" in why for why in r.reasons)
43
44
45 def test_matching_baseline_passes(db, tmp_path):
46     p = str(tmp_path / "b.json")
47     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
48     r = regression.regress(db, "vid1", GT, baseline_path=p)
49     assert r.regressed is False
50
51
52 def test_recall_drop_beyond_tolerance_regresses(tmp_path):
53     # DB has only ONE of the two GT rallies ? recall 0.5 vs baseline 1.0.
54     d = Database(str(tmp_path / "t.db"))
55     d.add_video("vid1", "/x/m.mp4", "processed", [])
56     d.add_play_segment("vid1", 0.0, 10.0)
57     p = str(tmp_path / "b.json")
58     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
59
60     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
61     assert r.regressed is True
62     assert r.recall == pytest.approx(0.5)
63     assert any("recall" in why for why in r.reasons)
64
65
66 def test_within_tolerance_does_not_regress(tmp_path):
67     d = Database(str(tmp_path / "t.db"))
68     d.add_video("vid1", "/x/m.mp4", "processed", [])
69     d.add_play_segment("vid1", 0.0, 10.0)
70     d.add_play_segment("vid1", 20.0, 30.0)
71     p = str(tmp_path / "b.json")
72     # Baseline only 0.02 above current perfect score ? within default 0.05 tolerance.
73     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
74     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
75     assert r.regressed is False
76
77
78 # ----- provider-driven end-to-end (uses the GS-1 FileProvider) -----
79
80 def test_regress_with_provider_file_tier(db, tmp_path):
81     csv = tmp_path / "gt.csv"
82     csv.write_text("start,end\n0,10\n20,30\n")
83     r = regression.regress_with_provider(
84         db, video_id="vid1", video_ref=str(csv), tier="file",
85         baseline_path=str(tmp_path / "b.json"),
86     )
87     assert r.precision == 1.0 and r.recall == 1.0
88
89
90 # ----- CLI exit-code contract (CI gating) -----
91
92 def test_cli_update_baseline_then_clean_run(db, tmp_path, capsys, monkeypatch):
93     # Point the runner at our temp DB via --db; CSV ground truth via 'file' tier.
94     csv = tmp_path / "gt.csv"
95     csv.write_text("start,end\n0,10\n20,30\n")
96     base = str(tmp_path / "b.json")
97     db_path = db.db_path if hasattr(db, "db_path") else str(tmp_path / "t.db")
98
99     # First, snapshot the baseline ? exit 0.
100    rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
101                             "--annotations", str(csv), "--db", db_path,
102                             "--baseline", base, "--update-baseline"])
103    assert rc == 0
104    assert "REGRESSED" not in capsys.readouterr().out
105

```

```

106
107 def test_cli_regression_exits_nonzero(tmp_path, capsys):
108     # DB missing one rally vs a baseline of perfect ? regression ? exit 1.
109     d = Database(str(tmp_path / "t.db"))
110     d.add_video("vid1", "/x/m.mp4", "processed", [])
111     d.add_play_segment("vid1", 0.0, 10.0)
112     csv = tmp_path / "gt.csv"
113     csv.write_text("start,end\n0,10\n20,30\n")
114     base = str(tmp_path / "b.json")
115     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=base)
116
117     rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
118                             "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
119                             "--baseline", base])
120     assert rc == 1
121     assert "REGRESSED" in capsys.readouterr().out
122
123
124 def test_cli_missing_baseline_exits_3(db, tmp_path):
125     # No baseline + no --update/--allow ? the gate cannot verify ? exit 3 (not a silent
126     pass).
127     csv = tmp_path / "gt.csv"
128     csv.write_text("start,end\n0,10\n20,30\n")
129     db_path = str(tmp_path / "t.db")
130     rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
131                             "--annotations", str(csv), "--db", db_path,
132                             "--baseline", str(tmp_path / "absent.json")])
133     assert rc == 3
134
135 def test_cli_missing_baseline_allowed_exits_0(db, tmp_path):
136     csv = tmp_path / "gt.csv"
137     csv.write_text("start,end\n0,10\n20,30\n")
138     rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
139                             "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
140                             "--baseline", str(tmp_path / "absent.json"),
141                             "--allow-missing-baseline"])
142     assert rc == 0
143
144
145 def test_iou_mismatch_is_incommensurable(db, tmp_path):
146     p = str(tmp_path / "b.json")
147     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
148                             gt_fingerprint=regression.gt_hash(GT))
149     # Same labels but scored at a different IoU ? not a valid comparison.
150     r = regression.regress(db, "vid1", GT, iou_threshold=0.3, baseline_path=p)

```

37. user (2026-06-21T02:35:06.001Z)

```

1     """Golden Regression Fixtures (M1) ? characterization tests over the committed synthetic
Tier-0.
2
3     Parametrized over ``load_manifest()`` so every committed fixture is exercised. The gate
is
4     **parse-compare** (never byte-compare), **CONTROL-only** (pure stdlib, no numpy on the
replay
5     path), and asserts the Tier-0 privacy invariant positively (no
person/audio/pose/gait/face keys).
6
7     $6 honest-limits caveat: green here proves only that the deterministic post-trajectory
transforms
8     are unchanged for the frozen synthetic cases ? characterization is behaviour-locking,
NOT
9     correctness, and synthetic-tier numbers measure plumbing, not field quality.
10    """

```

```

11     import json
12
13     import pytest
14
15     from backend.eval import golden_fixtures as gf
16
17     # Slugs discovered from the committed manifest ? the suite grows by accretion
    automatically.
18     MANIFEST = gf.load_manifest()
19     SLUGS = [entry["slug"] for entry in MANIFEST]
20
21
22     def test_manifest_is_non_empty():
23         """A committed manifest with at least one fixture must exist (else nothing is
    gated)."""
24         assert SLUGS, "no fixtures in eval_baselines/fixtures/manifest.json ? run --freeze-
    synthetic"
25
26
27     @pytest.mark.parametrize("slug", SLUGS)
28     def test_replay_matches_committed_expected(slug):
29         """Replay the fixture and parse-compare against its committed expected.json.
30
31         Characterization = behaviour-locking, NOT correctness (§6): a non-empty mismatch
    list means
32         the deterministic post-trajectory behaviour CHANGED for this frozen case, not that
    it is wrong.
33         """
34         committed = gf.load_fixture(slug)["expected"]
35         recomputed = gf.replay_fixture(slug)
36         mismatches = gf.compare_expected(recomputed, committed)
37         assert mismatches == [], (
38             f"{slug}: behaviour changed vs frozen snapshot (characterization, not
    correctness ? §6):\n"
39             + "\n".join(f" - {m}" for m in mismatches)
40         )
41
42
43     @pytest.mark.parametrize("slug", SLUGS)
44     def test_schema_version_le_current(slug):
45         """Every committed fixture's schema_version must be <= CURRENT_SCHEMA (forward-
    compatible)."""
46         fx = gf.load_fixture(slug)
47         assert int(fx["expected"]["schema_version"]) <= gf.CURRENT_SCHEMA
48         assert int(fx["provenance"]["schema_version"]) <= gf.CURRENT_SCHEMA
49
50
51     @pytest.mark.parametrize("slug", SLUGS)
52     def test_no_person_or_audio_keys(slug):
53         """Positive privacy assertion (D3 / §4c): NO biometric/identity/non-shuttle key
    appears
54         anywhere in the committed expected.json ? not trusting a silent 0.0 default."""
55         fdir = gf.FIXTURES_DIR / slug
56         raw = (fdir / "expected.json").read_text(encoding="utf-8").lower()
57         for forbidden in gf.FORBIDDEN_KEYS:
58             assert forbidden not in raw, (
59                 f"{slug}: forbidden key substring {forbidden!r} present in expected.json
    (Tier-0 leak)"
60             )
61         expected = gf.load_fixture(slug)["expected"]
62         for i, cand in enumerate(expected["candidates"]):
63             # The only feature block is the shuttle block, and it holds EXACTLY the 5
    shuttle cols.
64             assert set(cand.keys()) == {"start", "end", "shuttle"}, f"{slug}: candidate[{i}]
    has extra keys"

```

```

65         assert set(cand["shuttle"].keys()) == set(gf.SHUTTLE_FEATURES), (
66             f"{slug}: candidate[{i}].shuttle is not exactly the 5 shuttle features")
67
68
69     @pytest.mark.parametrize("slug", SLUGS)
70     def test_tier0_provenance_tags_by_kind(slug):
71         """Tier-0 hard tags, asserted BY KIND so the suite stays green when a
72         ``cleared_real`` fixture
73         is added (red-team #11: the old test hard-asserted kind=='synthetic' for ALL slugs,
74         which a
75         future real fixture would red). Every Tier-0 fixture must carry a license +
76         minors_free=True;
77         a synthetic one is additionally license=='synthetic', a cleared_real one is
78         owned==True."""
79         prov = gf.load_fixture(slug)["provenance"]
80         kind = prov["kind"]
81         assert kind in ("synthetic", "cleared_real"), f"unexpected Tier-0 kind {kind!r}"
82         # Common to ALL Tier-0 fixtures: non-blank license + minors excluded.
83         assert isinstance(prov.get("license"), str) and prov["license"].strip(),
84         "blank/missing license"
85         assert prov["minors_free"] is True
86         if kind == "synthetic":
87             assert prov["license"] == "synthetic"
88         else: # cleared_real
89             assert prov["owned"] is True
90             # Windowing pinned in the fixture (not the named SERVED constant) ? constraint 3.
91             assert set(prov["windowing"]) == {"vt", "mg", "mwd", "max_win"}
92
93
94     def test_control_is_not_learned():
95         """The replay must use the pure-stdlib CONTROL variant (no numpy/LogisticRegression
96         ? D5)."""
97         from backend.eval.experiment import CONTROL
98         assert CONTROL.is_learned() is False
99
100
101     def test_perturbing_trajectory_changes_segments(tmp_path):
102         """Mutating a copy of a trajectory and replaying yields DIFFERENT segments ? proving
103         the
104         windows are RE-DERIVED from the CSV each run, not merely echoed from
105         expected.json."""
106         slug = SLUGS[0]
107         src = gf.FIXTURES_DIR / slug
108         work = tmp_path / slug
109         work.mkdir()
110         # Copy the fixture, then TRUNCATE the trajectory to its first half so the derived
111         window's
112         # `end` shrinks ? a guaranteed structural change in the re-derived segments.
113         (Deleting an
114         # interior hole would not move start/end, since merge_gap coalesces across it and
115         the frame
116         # indices of the surviving rows are unchanged.)
117         lines = (src / "trajectory.csv").read_text(encoding="utf-8").splitlines()
118         header, body = lines[0], lines[1:]
119         mutated = [header] + body[: len(body) // 2]
120         (work / "trajectory.csv").write_text("\n".join(mutated) + "\n", encoding="utf-8",
121         newline="\n")
122         (work / "labels.csv").write_text((src / "labels.csv").read_text(encoding="utf-8"),
123         encoding="utf-8", newline="\n")
124         (work / "provenance.json").write_text((src /
125         "provenance.json").read_text(encoding="utf-8"),
126         encoding="utf-8", newline="\n")
127
128         original = gf.load_fixture(slug)["expected"]
129         perturbed = gf.replay_fixture(work)

```

```

117         assert perturbed["control_segments"] != original["control_segments"], (
118             "perturbed trajectory produced identical segments ? replay is echoing, not re-
deriving")
119
120
121     def test_freeze_is_idempotent(tmp_path):
122         """Re-freezing every built-in fixture into a temp dir produces expected.json
structurally
123         equal to the committed one (correct-by-construction + deterministic)."""
124         slugs = gf.freeze_synthetic(fixtures_dir=tmp_path)
125         assert slugs, "freeze_synthetic produced no fixtures"
126         for slug in slugs:
127             committed = gf.load_fixture(slug)["expected"]
128             refrozen = json.loads((tmp_path / slug /
"expected.json").read_text(encoding="utf-8"))
129             # Structural (parsed) equality ? no float churn, exact same shape/values.
130             assert gf.compare_expected(refrozen, committed) == [], (
131                 f"{slug}: re-frozen expected.json differs from committed (non-idempotent
freeze)")
132             assert refrozen == committed, f"{slug}: re-frozen expected.json not structurally
identical"
133
134
135     def test_check_cli_passes(capsys):
136         """The --check CLI replays all fixtures and exits 0 when they match the committed
snapshots."""
137         rc = gf.main(["--check"])
138         assert rc == 0
139

```

38. assistant (2026-06-21T02:35:08.798Z)

Now let me look at a few more test examples to understand the pattern better:

39. user (2026-06-21T02:35:09.548Z)

```

1     import os
2
3     from backend.eval import batch
4
5
6     # ---- path resolution -----
7
8     def test_resolve_relative_windows_path():
9         got = batch.resolve_video_path("./data/videos\\shuttle1.mp4", "/collector")
10        assert got == os.path.normpath("/collector/data/videos/shuttle1.mp4")
11
12
13    def test_resolve_absolute_path_kept():
14        got = batch.resolve_video_path("C:/vids/x.mp4", "/collector")
15        assert got == os.path.normpath("C:/vids/x.mp4")
16
17
18    def test_resolve_none():
19        assert batch.resolve_video_path(None, "/collector") is None
20        assert batch.resolve_video_path("", "/collector") is None
21
22
23    # ---- stage selection -----
24
25    def test_auto_stage_final_only_for_motion():
26        assert batch.default_stages_for("motion", "auto") == ["final"]
27        assert batch.default_stages_for("gemini", "auto") == ["final"]
28
29

```

```

30 def test_auto_stage_both_for_two_stage_detector():
31     assert batch.default_stages_for("yolo_hybrid", "auto") == ["candidates", "final"]
32
33
34 def test_explicit_stage_overrides_auto():
35     assert batch.default_stages_for("motion", "both") == ["candidates", "final"]
36     assert batch.default_stages_for("yolo_hybrid", "final") == ["final"]
37
38
39 # ---- aggregation -----
40
41 def _vid(tp, fp, fn, miou, f1):
42     return {"stages": {"final": {"score": {
43         "true_positives": tp, "false_positives": fp, "false_negatives": fn,
44         "mean_iou": miou, "f1": f1,
45     }}}}}
46
47
48 def test_aggregate_micro_and_macro():
49     per_video = [
50         _vid(tp=8, fp=2, fn=2, miou=0.9, f1=0.8), # P=.8 R=.8 F1=.8
51         _vid(tp=6, fp=0, fn=4, miou=0.8, f1=0.75), # P=1 R=.6
52     ]
53     agg = batch.aggregate(per_video, ["final"])[["final"]]
54     assert agg["tp"] == 14 and agg["fp"] == 2 and agg["fn"] == 6
55     assert agg["videos"] == 2
56     # micro precision = 14/16, recall = 14/20
57     assert round(agg["micro_precision"], 3) == round(14 / 16, 3)
58     assert round(agg["micro_recall"], 3) == round(14 / 20, 3)
59     # TP-weighted mean IoU = (0.9*8 + 0.8*6)/14
60     assert round(agg["mean_iou"], 4) == round((0.9 * 8 + 0.8 * 6) / 14, 4)
61     # macro F1 = mean of per-video F1
62     assert round(agg["macro_f1"], 4) == round((0.8 + 0.75) / 2, 4)
63
64
65 def test_aggregate_handles_missing_stage_and_zero_division():
66     agg = batch.aggregate([], ["final"])[["final"]]
67     assert agg["tp"] == 0 and agg["micro_f1"] == 0.0 and agg["mean_iou"] == 0.0
68
69

```

40. user (2026-06-21T02:35:09.858Z)

```

1     """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3     A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4     CSV with no GPU, no WSL, no model weights ? so the whole trajectory-hybrid pipeline
5     (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6     container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7     (``use_gpu=False``); this closes the last GPU/WSL-only gap so the entire pipeline is
8     runnable ? and regression-gateable ? without a GPU.
9
10    Two modes:
11    * replay ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12      pipeline with a committed/golden trajectory; deterministic + cross-OS).
13    * synth (default) ? generate a deterministic synthetic trajectory of
14      ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
15      gaps, so windowing yields a stable, known number of candidate windows.
16
17    This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18    "Optional Linux-native path") and the seed of the future ``LinuxNativeRunner``
19    (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20    ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21    WSL runners stay the default, so production behaviour is unchanged.
22    """
23    import os

```



```

24     import csv as _csv
25     import shutil
26     import logging
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.detectors.base import DetectorRunner
31     # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     @dataclass
38     class StubConfig:
39         """Settings for the CPU stub runner (built from ``config.json`` ->
40         ``indexing.stub``)."""
41         fps: float = 30.0
42         frame_width: float = 1280.0
43         num_rallies: int = 2
44         rally_seconds: float = 4.0
45         gap_seconds: float = 3.0          # dead gap between rallies (> merge_gap so
46         windows don't merge)
47         lead_seconds: float = 2.0        # dead lead-in
48         step_px: float = 12.0           # per-frame shuttle displacement during a rally
49         (>> velocity_thresh)
50         replay_csv: Optional[str] = None # if set + exists, emit this CSV verbatim instead
51         of synthesising
52
53     @classmethod
54     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
55         s = (idx_cfg or {}).get("stub", {}) or {}
56         return cls(
57             fps=float(s.get("fps", 30.0)),
58             frame_width=float(s.get("frame_width", 1280.0)),
59             num_rallies=int(s.get("num_rallies", 2)),
60             rally_seconds=float(s.get("rally_seconds", 4.0)),
61             gap_seconds=float(s.get("gap_seconds", 3.0)),
62             lead_seconds=float(s.get("lead_seconds", 2.0)),
63             step_px=float(s.get("step_px", 12.0)),
64             replay_csv=s.get("replay_csv"),
65         )
66
67     class StubDetectorRunner(DetectorRunner):
68         """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""
69
70         def __init__(self, cfg: Optional[StubConfig] = None):
71             self.cfg = cfg or StubConfig()
72
73         def healthcheck(self) -> Tuple[bool, str]:
74             return True, "stub CPU detector (no GPU/WSL)"
75
76         def _rows(self) -> List[Tuple[int, int, float, float]]:
77             """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight
78             bursts
79             separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
80             ?
81             well above the default 0.02 velocity threshold ? so every rally frame is
82             "active"
83             and each burst becomes exactly one candidate window."""
84             cfg = self.cfg
85             rows: List[Tuple[int, int, float, float]] = []
86             frame = 0
87             y = cfg.frame_width / 4.0

```

```

82         base_x = cfg.frame_width / 2.0
83         for _ in range(int(cfg.lead_seconds * cfg.fps)):
84             rows.append((frame, 0, 0.0, 0.0))
85             frame += 1
86         for r in range(cfg.num_rallies):
87             for i in range(int(cfg.rally_seconds * cfg.fps)):
88                 x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
89                 rows.append((frame, 1, x, y))
90                 frame += 1
91             if r != cfg.num_rallies - 1:
92                 for _ in range(int(cfg.gap_seconds * cfg.fps)):
93                     rows.append((frame, 0, 0.0, 0.0))
94                     frame += 1
95         return rows
96
97     def run_predict(self, video_win_path: str, output_win_dir: str,
98                    video_id: Optional[str] = None) -> Optional[str]:
99         """Write a deterministic trajectory CSV and return its path (never touches the
GPU/WSL)."""
100         os.makedirs(output_win_dir, exist_ok=True)

```

41. user (2026-06-21T02:35:10.448Z)

No files found

42. assistant (2026-06-21T02:35:13.612Z)

Let me search for storage fakes more systematically:

43. user (2026-06-21T02:35:14.262Z)

Found 12 files
tests\test_api_endpoints.py
tests\test_generate_highlights.py
tests\test_storage.py
tests\test_worker.py
artifacts\so.py
tests\test_compiler.py
tests\test_wsl_tilde_quoting.py
tests\test_fusion_golden_telemetry.py
tests\test_run_telemetry.py
tests\test_watermark.py
tests\test_detector_keying.py
tests\test_automated_provider.py

44. user (2026-06-21T02:35:14.664Z)

```

1     """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3     One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4     only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5     ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7     Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8     style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9     / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10    (tests pass a fake; production lets the constructor build one via boto3).
11    """
12    from __future__ import annotations
13
14    import os

```

```

15     from typing import Any, Iterator, Optional
16
17     from backend.storage.base import StorageBackend
18     from backend.storage.ref import StorageRef, StorageStat
19
20
21     class S3Storage(StorageBackend):
22         def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                     region: Optional[str] = None, addressing_style: str = "auto",
24                     signed_url_ttl: int = 3600, **_ignored) -> None:
25             self._ttl = signed_url_ttl
26             if client is not None:
27                 self._client = client
28             return
29             import boto3 # type: ignore # lazy: only when an s3:// URI is actually used
30             from botocore.config import Config # type: ignore
31             self._client = boto3.client(
32                 "s3",
33                 endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
34                 region_name=region,
35                 config=Config(s3={"addressing_style": addressing_style}),
36             )
37
38         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
39             if dst is None:
40                 raise ValueError("S3Storage.fetch_to_local requires a dst path")
41             dst = str(dst)
42             if os.path.exists(dst) and not overwrite:
43                 return dst
44             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
45             self._client.download_file(ref.bucket, ref.key, dst)
46             return dst
47
48         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
49             extra = {"ContentType": content_type} if content_type else None
50             self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
51             return ref
52
53         def exists(self, ref) -> bool:
54             try:
55                 self._client.head_object(Bucket=ref.bucket, Key=ref.key)
56                 return True
57             except Exception: # noqa: BLE001 - any boto/client error means "not found /
unreachable"
58                 return False
59
60         def stat(self, ref) -> StorageStat:
61             h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
62             return StorageStat(size=int(h.get("ContentLength", 0)),
modified=h.get("LastModified"),
63                             etag=h.get("ETag"), content_type=h.get("ContentType"))
64
65         def list(self, prefix) -> Iterator[StorageRef]:
66             paginator = self._client.get_paginator("list_objects_v2")
67             for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
68                 for obj in page.get("Contents", []) or []:
69                     key = obj["Key"]
70                     yield StorageRef("s3", prefix.bucket, key,
f"s3://{prefix.bucket}/{key}")
71
72         def delete(self, ref) -> None:
73             self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
74
75         def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
76             op = "get_object" if method.upper() == "GET" else "put_object"

```

```

77         return self._client.generate_presigned_url(
78             op, Params={"Bucket": ref.bucket, "Key": ref.key}, ExpiresIn=ttl or
self._ttl)
79

```

45. assistant (2026-06-21T02:35:16.537Z)

Now let me check how the storage tests inject fakes:

46. user (2026-06-21T02:35:17.040Z)

```

100     def test_gdrive_not_mounted_raises_with_remedy(monkeypatch):
101         # The scheme parses; if Drive isn't mounted (no auto-detected/configured root)
construction
102         # is a clear, actionable error ? install link + how to include the folder in syncing
+ the
103         # mount-root override ? rather than a cryptic failure downstream.
104         monkeypatch.delenv("GDRIVE_MOUNT_ROOT", raising=False)
105         assert parse_uri("gdrive://Sharing/x.mp4").backend == "gdrive"
106         with pytest.raises(RuntimeError) as ei:
107             # An explicit, non-existent root forces the error regardless of any real local
mount.
108             get_storage("gdrive", config={"gdrive": {"mount_root": "/no-such/Drive/My
Drive"}})
109             msg = str(ei.value)
110             assert "drive.google.com" in msg or "google.com/drive" in msg    # install pointer
111             assert "GDRIVE_MOUNT_ROOT" in msg                               # how to point at
the mount
112             assert "shortcut" in msg.lower() and "stream" in msg.lower()    # how to sync the
folder
113
114
115     def test_gdrive_mounted_but_unsynced_path_raises_with_remedy(tmp_path):
116         # Mount exists but the requested file isn't there yet (folder not synced) -> a
FileNotFoundError
117         # that names the fix (include the folder in syncing) instead of a bare "No such
file".
118         be = GDriveStorage(mount_root=str(_mount(tmp_path)))
119         with pytest.raises(FileNotFoundError) as ei:
120             be.fetch_to_local(parse_uri("gdrive://Sharing/not-synced.mp4"), str(tmp_path /
"out.mp4"))
121         msg = str(ei.value).lower()
122         assert "not present locally" in msg and "shortcut" in msg and "stream" in msg
123
124
125     # ----- S3 (fake client)
126
127     class _FakeS3Client:
128         def __init__(self):
129             self.store = {}
130
131         def upload_file(self, src, bucket, key, ExtraArgs=None):
132             with open(src, "rb") as f:
133                 self.store[(bucket, key)] = f.read()
134
135         def download_file(self, bucket, key, dst):
136             with open(dst, "wb") as f:
137                 f.write(self.store[(bucket, key)])
138
139         def head_object(self, Bucket, Key):
140             if (Bucket, Key) not in self.store:
141                 raise KeyError("404")
142             return {"ContentLength": len(self.store[(Bucket, Key)]), "ETag": '"e"',
143                     "ContentType": "video/mp4"}
144

```

```

145     def get_paginator(self, _op):
146         store = self.store
147
148         class _P:
149             def paginate(self, Bucket, Prefix=""):
150                 yield {"Contents": [{"Key": k} for (b, k) in store
151                                if b == Bucket and k.startswith(Prefix)]}
152         return _P()
153
154     def delete_object(self, Bucket, Key):
155         self.store.pop((Bucket, Key), None)
156
157     def generate_presigned_url(self, op, Params, ExpiresIn):
158         return f"https://s3.example/{Params['Bucket']}/{Params['Key']}?X-Amz-
Expires={ExpiresIn}"
159
160
161     def test_s3_backend_roundtrip_with_fake_client(tmp_path):
162         be = S3Storage(client=_FakeS3Client())
163         assert isinstance(be, StorageBackend)
164         src = tmp_path / "m.mp4"
165         src.write_bytes(b"VIDEO")
166         ref = parse_uri("s3://rally/videos/m.mp4")
167         be.put(str(src), ref, content_type="video/mp4")
168         assert be.exists(ref)
169         assert be.stat(ref).size == 5
170         dst = tmp_path / "dl" / "m.mp4"
171         assert be.fetch_to_local(ref, str(dst)) == str(dst)
172         assert dst.read_bytes() == b"VIDEO"
173         assert [r.uri for r in be.list(parse_uri("s3://rally/videos/"))] ==
["s3://rally/videos/m.mp4"]
174         url = be.signed_url(ref, ttl=900)
175         assert url.startswith("https://s3.example/rally/videos/m.mp4") and "900" in url
176         be.delete(ref)
177         assert not be.exists(ref)
178
179
180     def test_s3_exists_false_on_missing():
181         assert S3Storage(client=_FakeS3Client()).exists(parse_uri("s3://b/none")) is False
182
183
184     # ----- GCS (fake client)
185
186     class _FakeBlob:
187         def __init__(self, store, bucket, name):
188             self.store, self.bucket, self.name = store, bucket, name
189             self.size = self.updated = self.etag = self.content_type = None
190
191         def upload_from_filename(self, src, content_type=None):
192             with open(src, "rb") as f:
193                 self.store[(self.bucket, self.name)] = f.read()
194             self.content_type = content_type
195
196         def download_to_filename(self, dst):
197             with open(dst, "wb") as f:
198                 f.write(self.store[(self.bucket, self.name)])
199
200         def exists(self):
201             return (self.bucket, self.name) in self.store
202
203         def reload(self):
204             self.size = len(self.store[(self.bucket, self.name)])
205
206         def delete(self):
207             self.store.pop((self.bucket, self.name), None)

```

```

208
209     def generate_signed_url(self, version, expiration, method):
210         return f"https://gcs.example/{self.bucket}/{self.name}?version={version}"
211
212
213     class _FakeGCSClient:
214         def __init__(self):
215             self.store = {}
216
217         def bucket(self, name):
218             return type("_B", (), {"blob": lambda _self, key: _FakeBlob(self.store, name,
key)}})( )
219
220         def list_blobs(self, bucket, prefix=""):
221             return [_FakeBlob(self.store, bucket, k) for (b, k) in self.store
222                     if b == bucket and k.startswith(prefix)]
223
224
225     def test_gcs_backend_roundtrip_with_fake_client(tmp_path):
226         be = GCSStorage(client=_FakeGCSClient())
227         src = tmp_path / "w.json"
228         src.write_text("{}")
229         ref = parse_uri("gcs://rally/weights/w.json")
230         be.put(str(src), ref)
231         assert be.exists(ref)
232         assert be.stat(ref).size == 2
233         dst = tmp_path / "dl" / "w.json"
234         be.fetch_to_local(ref, str(dst))
235         assert dst.read_text() == "{}"
236         assert [r.uri for r in be.list(parse_uri("gcs://rally/weights/"))] ==
["gcs://rally/weights/w.json"]
237         assert be.signed_url(ref).startswith("https://gcs.example/rally/weights/w.json")
238         be.delete(ref)
239         assert not be.exists(ref)
240
241
242     # ----- Google Drive (local
mount)
243     # The backend reads a locally MOUNTED Google Drive (Google Drive for Desktop). A
tmp_path
244     # folder stands in for the mount's "My Drive" root ? no network, no Drive API, no extra
deps.
245
246     def _mount(tmp_path):
247         root = tmp_path / "My Drive"
248         (root / "Sharing").mkdir(parents=True)
249         return root

```

47. assistant (2026-06-21T02:35:18.971Z)

Excellent! Now let me look for information about what kind of profiles/variants exist:

48. user (2026-06-21T02:35:20.183Z)

Found 19 files
backend\eval\golden_real_fixtures.py
backend\eval\golden_fixtures.py
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
backend\eval\experiment.py
backend\eval\ablation.py
backend\eval\ablation_pdf.py
backend\eval>windowing.py
backend\eval\serve_contrast.py
backend\eval\distill_local.py

```
backend\eval\per_user_regression.py
backend\eval\per_user_eval.py
backend\eval\fusion_golden.py
backend\eval\score_calibration.py
backend\eval\eval_stats.py
backend\eval\rally_seq_proto.py
backend\eval\gemini_refine.py
backend\eval\training.py
backend\eval\classifier.py
```

49. user (2026-06-21T02:35:20.265Z)

```
1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. No new scoring math lives
7      here; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`   ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19     signals, where are the rallies). A Variant is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import glob
41     import json
42     import logging
43     import os
44     from dataclasses import dataclass, field
45     from datetime import datetime, timezone
46     from hashlib import sha1
47     from typing import Any, Callable, Dict, List, Optional, Sequence
48
49     from backend.eval.calibration import pct_improvement
50     from backend.eval.classifier import LogisticRegression
51     from backend.eval.gemini_refine import merge_overlaps
52     from backend.eval.metrics import Interval, match_intervals, score
53     from backend.eval.regression import gt_hash
```

```

54     from backend.eval.training import label_candidates
55     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
56     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
ratio
57     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
scores
58
59     logger = logging.getLogger(__name__)
60
61     # Where a comparison run appends one reproducible record. Tracked location (NOT under
62     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
63     # regression.DEFAULT_BASELINE_PATH.
64     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
65     _METRICS = ("precision", "recall", "f1", "mean_iou")
66
67
68     class ExperimentError(ValueError):
69         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70         score an unlabeled video with no pinned model, or a gate referencing an absent
71         feature)."""
72
73
74     # ----- Variant
75
76
77     @dataclass
78     class Variant:
79         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
80
81         A variant turns one video's persisted candidate windows + features into rally
82         intervals. Every knob defaults to the control behaviour (no gate, no learned
83         filter), so a variant that merely carries a new, disabled cue is byte-identical
84         to control ? the core no-regression guarantee.
85
86         Fields:
87         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
88           for the leaderboard and for selecting the served path (the existing
89           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
90           New cues are recorded here default-OFF.
91         - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
92           ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
93           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
94           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
95         - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
96           whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
97           cue persists that feature).
98         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
99           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
100           required for ``compare_unlabeled`` on a learned variant.
101         - ``feature_names`` ? the persisted features the learned filter consumes. When set
102           WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
103           (honest) ? the recipe, not a pinned artifact.
104         - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
105           overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
106         """
107
108         name: str
109         segmenter: str = "wasb_hybrid"
110         config_overrides: Dict[str, Any] = field(default_factory=dict)
111         cue_flags: Dict[str, bool] = field(default_factory=dict)
112         min_crossings: int = 0
113         min_players: int = 0
114         classifier_model: Optional[str] = None
115         feature_names: Optional[List[str]] = None

```



```

116         threshold: float = 0.5
117         merge_gap: float = 1.0
118         # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
119         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
120         # failure where a fixed 0.5 zeroed out a small-candidate video.
121         tune_threshold: bool = False          # pick the F1-best threshold on the train fold
122         calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
123
124         def is_learned(self) -> bool:
125             """True if this variant applies a learned classifier (pinned model or
recipe)."""
126             return self.classifier_model is not None or bool(self.feature_names)
127
128         def to_dict(self) -> dict:
129             return {
130                 "name": self.name,
131                 "segmenter": self.segmenter,
132                 "config_overrides": self.config_overrides,
133                 "cue_flags": self.cue_flags,
134                 "min_crossings": self.min_crossings,
135                 "min_players": self.min_players,
136                 "classifier_model": self.classifier_model,
137                 "feature_names": self.feature_names,
138                 "threshold": self.threshold,
139                 "merge_gap": self.merge_gap,
140                 "tune_threshold": self.tune_threshold,
141                 "calibrate": self.calibrate,
142             }
143
144         @classmethod
145         def from_dict(cls, d: dict) -> "Variant":
146             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
147             return cls(**{k: v for k, v in d.items() if k in known})
148
149         def spec_hash(self) -> str:
150             """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard

```

50. assistant (2026-06-21T02:35:22.297Z)

Perfect! Now let me compile the final information. Let me check one more key piece - the CI lanes and what they test:

51. user (2026-06-21T02:35:23.044Z)

Found 4 files
tests\test_golden_fixtures_split.py
backend\eval\golden_real_fixtures.py
backend\eval\golden_fixtures.py
tests\test_golden_real_fixtures.py

52. assistant (2026-06-21T02:35:25.190Z)

Let me read test_golden_real_fixtures to understand the M2 refusal gate:

53. user (2026-06-21T02:35:25.664Z)

```

1         """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3         These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
an OWNED,

```

```

4      minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
fixture) and
5      its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
6
7      Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
NO GPU.
8      We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory``
and the
9      ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"
CSVs, then
10     prove:
11         * the refusal gate blocks every unsafe combination (M2);
12         * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no
FORBIDDEN_KEYS);
13         * the time-origin reset is metric-INVARIANT (score identical with/without; only
absolute
14         ``control_segments`` shift toward 0);
15         * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
16
17     §6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
18     are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
19     ""
20     import pytest
21
22     from backend.eval import golden_fixtures as gf
23
24
25     # -----
26     # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
27     # -----
28
29
30     def _write_source(tmp_path, *, frame_offset: int = 0):
31         """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.
32
33         Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
34         ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
absolute
35         capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
36         ""
37         spec = gf.builtin_specs()[0] # clean_single_rally
38         fps, fw = spec.fps, spec.frame_width
39         rows = gf.make_synthetic_trajectory(spec)
40         if frame_offset:
41             rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
42         gts = list(spec.gts)
43         if frame_offset:
44             off_sec = frame_offset / fps
45             gts = [(s + off_sec, e + off_sec) for (s, e) in gts]
46
47         traj = tmp_path / "src_trajectory.csv"
48         labels = tmp_path / "src_labels.csv"
49         gf.write_trajectory_csv(rows, traj)
50         gf.write_labels_csv(gts, labels)
51         return traj, labels, fps, fw, gts
52
53
54     # -----
55     # M2 ? refusal gate
56     # -----
57
58
59     @pytest.mark.parametrize(

```

```

60         "label, kwargs, why",
61         [
62             ("scenario", {"minors_free": False, "owned": True, "license": "owned"},
63              "minors_free=False"),
64             ("scenario", {"minors_free": True, "owned": False, "license": "owned"},
65              "owned=False"),
66             ("scenario", {"minors_free": True, "owned": True, "license": ""}, "license=''),
67             ("scenario", {"minors_free": True, "owned": True, "license": "   "},
68              "license=blank"),
69             ("scenario", {"minors_free": True, "owned": True, "license": None},
70              "license=None"),
71             # red-team #1: a free-text identifier label (name / event / spaces /
72             punctuation) is refused.
73             ("Lin Dan vs Lee, India Open 2024",
74              {"minors_free": True, "owned": True, "license": "owned"}, "label=player-name
75              free text"),
76             ("mens singles final",
77              {"minors_free": True, "owned": True, "license": "owned"}, "label=spaces"),
78             ("Final_2024", {"minors_free": True, "owned": True, "license": "owned"},
79              "label=uppercase"),
80             ("", {"minors_free": True, "owned": True, "license": "owned"}, "label=empty"),
81         ],
82     )
83
84     def test_refusal_gate_blocks_unsafe(tmp_path, label, kwargs, why):
85         """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
86         blank license
87         AND a STRICT-slug label (no free-text identifier ? red-team #1).
88
89         The refusal is the code-enforced provenance gate (§4c / §7 ?): the unsafe public-
90         fixture path
91         must be impossible to reach silently. A refused freeze leaves NO partial fixture dir
92         (red-team
93         #3b: write-to-temp-then-promote).
94         """
95         traj, labels, fps, fw, _ = _write_source(tmp_path)
96         with pytest.raises(ValueError) as ei:
97             gf.freeze_real_fixture(
98                 label, traj, labels,
99                 fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
100         # RefusalError is a ValueError subclass; the message must name a reason.
101         assert isinstance(ei.value, gf.RefusalError), why
102         assert "REFUSED" in str(ei.value), why
103         # Nothing was written for a refused freeze (no fixture dir, partial or otherwise).
104         assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
105         "fixtures").iterdir()), (
106             f"{why}: a refused freeze must not write any fixture")
107
108         # -----
109         # P1 ? de-attribution
110         # -----
111
112     def test_real_fixture_is_deattributed(tmp_path):
113         """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed
114         surrogate slug,
115         no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
116         fixtures = tmp_path / "fixtures"
117         traj, labels, fps, fw, _ = _write_source(tmp_path)
118         expected = gf.freeze_real_fixture(
119             "clean_rally", traj, labels,
120             license="owned", minors_free=True, owned=True,
121             fps=fps, frame_width=fw, fixtures_dir=fixtures)
122         slug = expected["slug"]

```

```

113     assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
{slug!r}"
114
115     prov = gf._read_json(fixtures / slug / "provenance.json")
116     assert prov["kind"] == "cleared_real"
117     assert prov["minors_free"] is True and prov["owned"] is True
118     assert prov["license"] == "owned"
119     assert prov["slug"] == slug
120     assert prov["label"] == "clean_rally" # strict-slug label is committed verbatim
121     # No attribution keys / no source filename anywhere in the provenance. `source_note`
is now
122     # DROPPED entirely (red-team #1b): operator free-text never reaches the committed
bytes.
123     for banned in ("video_id", "name", "filename", "path", "match", "capture_date",
"source_note"):
124         assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
125     prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
126     for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
127         assert tok not in prov_text, f"source token {tok!r} leaked into provenance"
128
129     # expected.json carries no biometric/identity/non-shuttle stream.
130     exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
131     for forbidden in gf.FORBIDDEN_KEYS:
132         assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"
133
134     # The fixture is registered in the manifest with kind=cleared_real.
135     manifest = gf.load_manifest(fixtures)
136     rows = [e for e in manifest if e["slug"] == slug]
137     assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"
138
139
140     def test_supplied_slug_is_honored(tmp_path):
141         """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
concern)."""
142         fixtures = tmp_path / "fixtures"
143         traj, labels, fps, fw, _ = _write_source(tmp_path)
144         expected = gf.freeze_real_fixture(
145             "scenario", traj, labels, slug="fx_custom00",
146             license="owned", minors_free=True, owned=True,
147             fps=fps, frame_width=fw, fixtures_dir=fixtures)
148         assert expected["slug"] == "fx_custom00"
149         assert (fixtures / "fx_custom00" / "expected.json").is_file()
150
151
152     def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
153         """Freezing a second real fixture must APPEND to the manifest, not wipe the
first."""
154         fixtures = tmp_path / "fixtures"
155         traj, labels, fps, fw, _ = _write_source(tmp_path)
156         e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
157             license="owned", minors_free=True, owned=True,
158             fps=fps, frame_width=fw, fixtures_dir=fixtures)
159         e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
160             license="owned", minors_free=True, owned=True,
161             fps=fps, frame_width=fw, fixtures_dir=fixtures)
162         slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
163         assert {e1["slug"], e2["slug"]} <= slugs
164
165
166     def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
167         """source_note is ACCEPTED-BUT-NOT-PERSISTED (red-team #1b), yet still scanned
defensively ? a

```

```

168         path/name in it trips the positive scan and refuses (and leaves no partial dir, red-
team #3b)."""
169         fixtures = tmp_path / "fixtures"
170         traj, labels, fps, fw, _ = _write_source(tmp_path)
171         with pytest.raises(gf.RefusalError):
172             gf.freeze_real_fixture(
173                 "scenario", traj, labels, slug="fx_leak0001",
174                 license="owned", minors_free=True, owned=True,
175                 fps=fps, frame_width=fw, fixtures_dir=fixtures,
176                 source_note=str(traj)) # leaking the source path ? must be refused
177         # The refused freeze left no fixture dir behind (write-to-temp-then-promote).
178         assert not (fixtures / "fx_leak0001").exists(), "refused freeze must not leave a
partial dir"
179
180
181     def test_source_note_safe_is_not_persisted(tmp_path):
182         """A SAFE source_note (no path/name) is accepted but NEVER written to the committed
provenance
183         (red-team #1b: operator free-text is an attribution back-channel, dropped from
committed bytes)."""
184         fixtures = tmp_path / "fixtures"
185         traj, labels, fps, fw, _ = _write_source(tmp_path)
186         expected = gf.freeze_real_fixture(
187             "scenario", traj, labels, slug="fx_note0001",
188             license="owned", minors_free=True, owned=True,
189             fps=fps, frame_width=fw, fixtures_dir=fixtures,
190             source_note="benign generic note")
191         prov = gf._read_json(fixtures / expected["slug"] / "provenance.json")
192         assert "source_note" not in prov, "source_note must NOT be persisted into
provenance"
193         prov_text = (fixtures / expected["slug"] /
"provenance.json").read_text(encoding="utf-8")
194         assert "benign generic note" not in prov_text, "source_note text leaked into
committed bytes"
195
196
197     def test_freeze_real_label_slug_committed(tmp_path):
198         """A valid strict-slug label is committed verbatim; a free-text label is refused
(red-team #1a)."""
199         fixtures = tmp_path / "fixtures"
200         traj, labels, fps, fw, _ = _write_source(tmp_path)

```

54. assistant (2026-06-21T02:35:27.607Z)

Perfect! Now I have all the context I need. Let me compile the testing surface map:

55. user (2026-06-21T02:36:22.261Z)

Structured output provided successfully

56. assistant (2026-06-21T02:36:38.797Z)

Perfect! I've completed a comprehensive mapping of the testing surface for the badminton-highlight-indexer project. Here's a summary of what I found:

Testing Surface Map ? Key Findings

Core Testing Infrastructure

****run_all_tests.py**** is the single entry point: passes all pytest args through, defaults to quiet mode unless `-v/-q` specified, fully mocked (no GPU/WSL/network), optional obsreport skip-gates reporting tests.

CI Lanes (4 production gates)

1. ****Lint**** ? ruff (pyflakes/bugbear/pycodestyle)
2. ****Typecheck**** ? mypy with lenient baseline + explicit quarantine list
3. ****Import-smoke**** ? syntax + GPU-free imports (catches module-level errors early)
4. ****Full-suite**** ? CPU-only torch, coverage floor 70%, plus ****golden-crossos**** matrix (ubuntu/windows/macOS replay CONTROL fixtures with parse-compare, pure stdlib)

M0 ? Golden Fixtures (Tier-0 Regression Gate)

- ****Synthetic, deterministic, behavior-locking**** ? not correctness (\$6 caveat)
- 3 built-in scenarios: ``clean_single_rally``, ``multi_rally_with_deadtime``, ``occlusion_break``
- ****CONTROL-only replay**** (pure stdlib, no numpy execution) ? pinned windowing params (`vt=0.02`, `mg=2.0`, `mwd=2.0`)
- ****Parse-compare**** (never byte-compare): exact ints/structure, abs-tolerance floats (`1e-6`)
- ****Tier-0 privacy invariant****: no person/pose/gait/face/audio keys (FORBIDDEN_KEYS tuple)
- Parametrized tests over ``load_manifest()`` ? committed fixtures drive suite growth

M2/P1 ? Real Fixture De-Attribution & Refusal Gate

- ****RefusalError**** blocks unsafe: requires `minors_free=True`, `owned=True`, non-blank license, strict-slug label
- ****source_note**** accepted-but-not-persisted (red-team: operator free-text never reaches committed bytes)
- ****Surrogate fx_hex slugs**** + `kind='cleared_real'` provenance tags
- ****Time-origin reset metric-invariant**** (score unchanged, segments shift toward 0)
- ****Positive privacy assertion****: scans for FORBIDDEN_KEYS in `expected.json` + source leaks in provenance
- Write-to-temp-then-promote pattern (no partial dirs on refusal)

M3.5 ? Stub Detector Seam (GPU-Free E2E)

- CPU-only ``StubDetectorRunner`` produces deterministic `Frame,Visibility,X,Y` CSV
- Dual mode: synth (num_rallies bursts) or replay (golden CSV verbatim)
- Injected via ``detector_impl='stub'`` or ``RALLY_DETECTOR_IMPL=stub`` env (env wins)
- Feeds shared model-agnostic `TrackNetRunner` parse/window code
- Proves entire trajectory-hybrid pipeline runnable on CPU, deterministic + cross-OS

Storage Backend Fakes

- ****S3****: in-memory ``_FakeS3Client`` (dict-backed, no boto3)
- ****GCS****: ``_FakeBlob`` + ``_FakeGCSCClient`` (no google-cloud-storage SDK)
- ****GDrive****: reads locally MOUNTED Drive (Drive for Desktop), `tmp_path` stands in for mount
- All backends exercise same ``StorageBackend`` ABC (`fetch_to_local`, `put`, `exists`, `stat`, `list`, `delete`, `signed_url`)

Experiment Harness & Variant Profiles

- ****Variant**** dataclass as declarative re-scoring recipe (not a code branch)
- Fields: `name`, `segmenter`, `min_crossings` (Gate A), `min_players` (Gate B), `classifier_model`, `feature_names`, `threshold`, `merge_gap`, `tune_threshold`, `calibrate`
- ****CONTROL**** is blessed no-op (all defaults, byte-identical guarantee for disabled cues)
- Three modes: `compare` (labeled A/B + LOVO-honest), `compare_unlabeled` (SxS divergence), `promotion gate`
- `VideoCandidates(name, intervals, features, gts)` reused across variants

Regression Testing

- ``save_baseline()`` / ``load_baselines()`` ? human-readable JSON under `eval_baselines/`
- ``regress()`` checks precision/recall/f1/IoU against tolerance (default 0.05)
- CLI exits: 0=green, 1=regressed, 3=missing baseline
- ``batch.aggregate()`` computes micro/macro metrics across per-video results

Multi-Profile Proof Strategy

1. ****Golden-crossos**** (3 OSes) proves CONTROL determinism
2. ****Stub detector + storage fakes**** prove GPU-free E2E with injected clients
3. ****Experiment compare(CONTROL, disabled_cue)**** proves F1 delta ? 0 (no-regression invariant)
4. ****Regression baseline**** gates per-video + batch metrics, exits on deviation
5. ****Import smoke + lint + typecheck**** catch early errors (no GPU stack required)

The design is intentionally ****offline + mocked + deterministic****. All backends injectable, all variants reuse the same eval stack, configuration via minimalist env vars + config dicts + CLI

args. The architecture separates DETECTION (persisted signals) from DECISION (variant re-scoring), enabling pure-offline A/B/promotion gates with zero production risk.